CẨM NANG NGÀNH DEVOPS

Giáo trình kỹ thuật · bản đọc thử 11 chương

Chưa phải bản phát hành hoàn chỉnh

MỤC LỤC

Chương 1. Từ chương trình đến tiến trình 4
1.1. Ba lớp thường bị gọi chung là “ứng dụng” . . . 4
1.2. Một tiến trình ra đời và đổi chương trình . . . 4
1.3. Tên tệp, file descriptor và đối tượng đang mở . . . 5
1.4. Quyền và giới hạn: tiến trình không có toàn quyền chỉ vì nó chạy . . . 6
1.5. Từ tiến trình đến dịch vụ . . . 6
1.6. Áp mô hình vào một sự cố nhỏ . . . 7
Bài tập . . . 7
Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu? 8
2.1. Công việc, thời gian chạy và thời gian chờ . . . 8
2.2. Địa chỉ ảo không phải RAM đã dùng . . . 8
2.3. Page cache và lời hứa ghi dữ liệu . . . 9
2.4. Giới hạn tài nguyên và hành vi dưới tải . . . 9
2.5. Lần theo một phút tăng độ trễ . . . 10
Bài tập . . . 11
Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd 12
3.1. Từ mã người dùng đến lời gọi hệ thống . . . 12
3.2. Tên tệp, inode và bước đi qua thư mục . . . 12
3.3. Một unit biến tiến trình thành dịch vụ như thế nào . . . 13
3.4. Kiểm chứng trước và sau khi khởi động . . . 14
Bài tập . . . 15
Chương 4. Địa chỉ, định tuyến và TCP: một kết nối đi qua đâu? 16
4.1. Địa chỉ IP và prefix xác định phạm vi nào? . . . 16
4.2. Route chọn chặng kế tiếp; ARP tìm địa chỉ trên chặng đó . . . 16
4.3. TCP tạo một cuộc hội thoại, không tạo đường đi IP . . . 17
4.4. NAT đổi thông tin trên đường và cần đường về . . . 18
4.5. Lần theo một client không kết nối được . . . 18
Bài tập . . . 19
Chương 5. Từ tên miền đến phản hồi: DNS, TLS, HTTP và reverse proxy 21
5.1. DNS trả dữ liệu tên; client vẫn phải chọn đích . . . 21
5.2. TCP và TLS giải quyết hai câu hỏi khác nhau . . . 21
5.3. HTTP mang ý định; proxy quyết định chuyển tiếp . . . 22
5.4. Một sự cố, nhiều ranh giới cần đo . . . 24
Bài tập . . . 25
Chương 6. Git là đồ thị lịch sử, không phải thư mục sao lưu 26
6.1. Từ nội dung tệp tới commit có cha . . . 26
6.2. Ba trạng thái cần phân biệt trước khi commit . . . 26

6.3. Branch là ref; merge là kết quả của hai dòng lịch sử . . . 27
6.4. Rebase thay gốc bằng commit mới . . . 27
6.5. Revert thêm lịch sử; reset dịch vị trí . . . 28
6.6. Đọc chứng cứ trước khi chọn hành động . . . 28
Bài tập . . . 28
Chương 7. Bash và ranh giới lỗi trong tự động hóa 29
7.1. Shell biến dòng chữ thành lệnh như thế nào? . . . 29
7.2. Redirection, pipeline và exit status . . . 29
7.3. Vì sao `set -e` không phải bộ xử lý lỗi? . . . 30
7.4. Tín hiệu, trap và việc chạy lại . . . 30
7.5. Tình huống: backup báo thành công giả . . . 31
Bài tập . . . 32
Chương 8. Python và Go: điều phối tác vụ, giữ đúng ranh giới lỗi 34
8.1. Khi nào giữ Bash, khi nào dùng Python hoặc Go? . . . 34
8.2. Đối số, cấu hình và môi trường thực thi . . . 34
8.3. Một lỗi được quan sát qua Python và Go . . . 35
8.4. Timeout, cancellation và thất bại từng phần . . . 39
8.5. Đóng gói và kiểm chứng trước khi chạy ở nơi khác . . . 40
Bài tập . . . 40
Chương 9. Từ commit đến phục hồi một bản phát hành 41
9.1. Danh tính của bản được chạy . . . 41
9.2. Cổng kiểm thử và quyền phát hành . . . 41
9.3. Triển khai không đồng nghĩa sẵn sàng . . . 42
9.4. Sự cố xuyên suốt: bản chạy đã lùi, dữ liệu chưa lùi . . . 43
Bài tập . . . 43
Chương 10. Namespace, cgroup và tiến trình trong container 45
10.1. Một tiến trình có nhiều góc nhìn . . . 45
10.2. Cgroup v2 điều tiết tài nguyên . . . 46
10.3. Image được thực thi thành container như thế nào . . . 46
10.4. Một sự cố, bốn giả thuyết khác nhau . . . 47
Bài tập . . . 48
Chương 11. Image, registry, Compose và vòng đời dữ liệu 49
11.1. Từ bản build tới registry . . . 49
11.2. Container là lần chạy; mount quyết định nơi ghi . . . 49
11.3. Compose nối dịch vụ, mạng và nơi lưu . . . 50
11.4. Sự cố: cùng đường dẫn, ba nơi chứa dữ liệu . . . 51
Bài tập . . . 52
Lời giải 53

Chương 1. Từ chương trình đến tiến trình

Một dịch vụ có thể có mã nguồn đúng mà vẫn không phục vụ được yêu cầu. Tệp thực thi có thể tồn tại, nhưng chưa có tiến trình nào đang chạy; tiến trình có thể chạy, nhưng không mở được cổng; cổng có thể mở, nhưng tiến trình không đọc được tệp cấu hình. Để chẩn đoán những tình huống này, trước hết cần biết hệ điều hành đã biến chương trình thành một thực thể đang hoạt động như thế nào. Chương này xây dựng mô hình ấy trước khi nói đến container, điều phối hay giám sát.

1.1. Ba lớp thường bị gọi chung là “ứng dụng”

Mã nguồn là văn bản mô tả hành vi. Tệp thực thi hoặc chương trình được thông dịch là vật liệu để khởi chạy. Tiến trình là một lần thực thi cụ thể: nó có định danh, không gian địa chỉ, trạng thái xử lý, thông tin định danh người dùng và tập tài nguyên đang mở. Một chương trình có thể được chạy nhiều lần, tạo ra nhiều tiến trình độc lập. Ngược lại, một tiến trình có thể thay chương trình đang thực thi mà vẫn giữ định danh tiến trình, khi dùng cơ chế `execve`.

Sự phân biệt này giải thích vì sao “build thành công” không đồng nghĩa “dịch vụ sống”. Build xác nhận một số tính chất của artifact; nó không chứng minh hệ điều hành cấp đủ tài nguyên, địa chỉ mạng sẵn có hay cấu hình runtime hợp lệ. Đối với người vận hành, câu hỏi cần tách làm hai: artifact nào được đưa tới máy, và tiến trình nào đang thực thi artifact đó? Nếu hai câu trả lời khác nhau, một lần triển khai có thể đã thay tệp trên đĩa mà chưa thay phiên bản đang phục vụ.

CPU thực thi lệnh của tiến trình theo các khoảng thời gian; bộ nhớ chứa mã và dữ liệu cần cho lần chạy; nhân hệ điều hành quản lý tài nguyên và cung cấp các lời gọi hệ thống. Đó là mô hình khởi đầu, chưa phải toàn bộ chi tiết của Linux. Trong một tiến trình còn có thể có nhiều luồng thực thi cùng chia sẻ phần lớn không gian địa chỉ. Vì thế, số tiến trình không phải số công việc đồng thời, và mức dùng CPU không thể suy ra chỉ từ số PID.

1.2. Một tiến trình ra đời và đổi chương trình

Trên Linux, `fork` tạo tiến trình con. Tiến trình cha nhận PID của con, còn con nhận số 0; nếu không tạo được con, chỉ cha nhận `-1` và mã lỗi. Nhờ vậy hai tiến trình đi theo hai nhánh xử lý khác nhau. Không nên hình dung rằng toàn bộ bộ nhớ vật lý bị sao chép ngay lập tức: triển khai Linux dùng các trang copy-on-write, tạo bản riêng khi cần ghi. Đây là chi tiết triển khai Linux, không phải định nghĩa phổ quát của mọi hệ điều hành.

Sau bước tạo con, một cách khởi chạy chương trình khác là gọi `execve`. Lời gọi này thay process image hiện tại bằng chương trình mới; nó không tạo thêm một tiến trình mới. Nếu thành công, luồng lệnh cũ không tiếp tục ngay sau lời gọi. Nếu lỗi, lời gọi trả về `-1`, đặt `errno` và chương trình cũ còn cơ hội xử lý. Cặp cơ chế tạo con rồi thay chương trình giúp hiểu vì sao một shell có thể tiếp tục tồn tại sau khi khởi chạy lệnh khác, đồng thời vì sao tiến trình của lệnh có PID riêng.

Ví dụ chạy trên Linux bằng `python3 fork_exec.py`; thêm `fail` để cố ý chọn đường dẫn không tồn tại. PID do hệ điều hành cấp. Cha in trạng thái sau khi `waitpid` trả về.

```
"""Linux-only trace of fork and execve; run with optional argument 'fail'."""

import os
import sys


child = os.fork()
if child == 0:
    path = "/definitely/missing" if sys.argv[1:] == ["fail"] else "/bin/echo"
    os.write(2, f"child: fork returned {child}, PID {os.getpid()}\n".encode())
    try:
        os.execve(path, ["echo", "child: execve succeeded"], os.environ.copy())
    except OSError as exc:
        os.write(2, f"child: execve failed: {exc.strerror}\n".encode())
        os._exit(127)

print(f"parent: fork returned child PID {child}", flush=True)
_, status = os.waitpid(child, 0)
if os.WIFEXITED(status):
    print(f"parent: child exited with {os.WEXITSTATUS(status)}")
else:
    print("parent: child did not exit normally")
```

Ở cả hai lần chạy, `fork` thành công tạo hai PID. Trong cha, `child` là PID dương của con; trong con, `child` bằng 0. Nhánh thành công thay chương trình của con bằng `/bin/echo`, giữ nguyên PID của con và không vào khối `except`. Nhánh lỗi khiến lời gọi Python ném `OSError`; con in lỗi rồi thoát với mã 127 theo quy ước riêng của ví dụ, còn cha nhận mã ấy qua `waitpid`. Ở giao diện lời gọi hệ thống C, lỗi tương ứng là giá trị `-1` và `errno`; Python chuyển nó thành ngoại lệ. Nếu `fork` thất bại, Python cũng ném `OSError` và không tạo con để chờ. Mã 127 ở đây là lựa chọn của chương trình, không phải mã lỗi cố định do `execve` áp đặt.

Ví dụ chỉ dành cho tiến trình Python một luồng. Từ Python 3.12, `os.fork()` có thể phát `DeprecationWarning` nếu phát hiện tiến trình có nhiều luồng: các khóa do luồng khác giữ có thể khiến tiến trình con kẹt trước khi thay chương trình. Không dùng đoạn mã này làm mẫu khởi chạy trong ứng dụng Python đa luồng.

Mô hình này cũng giải thích một lỗi vận hành phổ biến: chương trình con kế thừa tài nguyên mà người gọi quên đóng. Theo mặc định, một file descriptor có thể còn mở qua `execve`; cờ close-on-exec thay đổi hành vi đó. Nếu vô tình giữ đầu ghi của một pipe ở tiến trình khác, đầu đọc có thể chưa nhìn thấy kết thúc luồng dù bên tạo dữ liệu đã xong. Kết luận này là hệ quả của vòng đời tài nguyên, không phải một quy tắc riêng của Bash.

1.3. Tên tệp, file descriptor và đối tượng đang mở

Khi tiến trình gọi `open`, nó nhận một số nguyên nhỏ không âm: file descriptor. Số này là chỉ số trong bảng descriptor của chính tiến trình. Descriptor không phải tên tệp, cũng không phải bản thân nội dung tệp. Nó tham chiếu tới một open file description do nhân quản lý, nơi giữ vị trí đọc/ghi hiện tại và các cờ trạng thái. Hai descriptor có thể cùng tham chiếu một open file description; khi đó chúng chia sẻ vị trí đọc/ghi.

Sự khác biệt có ý nghĩa khi đọc log. Nếu một tiến trình đã mở tệp log và người vận hành đổi tên hoặc xóa đường dẫn, descriptor đang mở vẫn có thể trỏ tới đối tượng cũ. Tạo một tệp mới ở cùng tên đường dẫn không buộc tiến trình chuyển đầu ghi sang tệp mới. Phần thực hành về log rotation ở các chương sau sẽ dựa trên quan hệ này để giải thích vì sao có trường hợp dung lượng đĩa chưa được giải phóng và vì sao cần yêu cầu dịch vụ mở lại log.

Ba descriptor quen thuộc khi khởi chạy chương trình là đầu vào chuẩn, đầu ra chuẩn và đầu lỗi chuẩn. Chuyển hướng trong shell thay đổi các luồng mà tiến trình nhận được; nó không sửa ý nghĩa của chương trình. Khi một dịch vụ ghi thông báo lỗi vào đầu ra chuẩn, người thu thập log vẫn có thể lấy được dữ liệu, nhưng việc phân loại mức độ lỗi trở nên khó hơn. Do vậy, quy ước đầu ra là một phần của hợp đồng vận hành.

1.4. Quyền và giới hạn: tiến trình không có toàn quyền chỉ vì nó chạy

Một tiến trình mang thông tin định danh và chịu kiểm tra quyền khi truy cập tài nguyên. Có quyền đọc một đường dẫn không đồng nghĩa có quyền thực thi tệp, và có quyền thực thi một tệp không đồng nghĩa được đọc mọi tệp mà chương trình sẽ mở sau đó. Nhìn từ bên ngoài, cả hai tình huống có thể hiện thành "ứng dụng không khởi động"; bên trong, điểm lỗi nằm ở những bước khác nhau. Về sau, mô hình này sẽ nối với quyền Linux, IAM và quyền của tài khoản dịch vụ trong Kubernetes.

Tiến trình còn hoạt động trong các giới hạn tài nguyên. Một lần tạo tiến trình con có thể thất bại khi đạt giới hạn số tác vụ; `fork` báo lỗi thay vì bảo đảm thành công. Vì vậy, một bộ điều phối không thể chữa một chương trình tạo tiến trình vô hạn chỉ bằng cách tự khởi động lại nó. Tự phục hồi là chính sách phản ứng với lỗi; ngăn lỗi lặp lại đòi sửa nguyên nhân hoặc đặt giới hạn thích hợp.

Lưu ý rằng quyền truy cập và khả năng nhìn thấy tài nguyên là hai câu hỏi khác nhau. Một namespace có thể cho tiến trình một góc nhìn riêng về một loại tài nguyên của hệ thống. Nhưng góc nhìn riêng không tự chứng minh rằng tiến trình an toàn hay không thể tác động đến thứ bên ngoài ranh giới đó. Ở phần container, ta sẽ kết hợp namespace với giới hạn tài nguyên, quyền và chính sách bảo mật thay vì xem "cách ly" là một thuộc tính nhị phân.

1.5. Từ tiến trình đến dịch vụ

Dịch vụ là một hợp đồng vận hành đặt lên tiến trình: khởi động khi nào, cấu hình từ đâu, báo trạng thái ra sao, dừng theo tín hiệu nào, có khởi động lại không và dữ liệu bền vững nằm ở đâu. Không phải mọi tiến trình đều là dịch vụ. Một công cụ chạy xong rồi thoát có vòng đời khác một HTTP server tồn tại lâu và tiếp nhận yêu cầu mới.

Khi một tiến trình thoát, hệ điều hành có thể ghi nhận mã kết thúc. Mã này cho biết chương trình kết thúc theo quy ước nào; tự nó chưa cho biết nguyên nhân gốc. Cùng một mã khác không có bảo đảm diễn giải giống nhau giữa mọi chương trình. Một hệ thống giám sát tốt cần xem thêm thời điểm, stderr, tình trạng tài nguyên và sự kiện từ bộ quản lý dịch vụ. Đọc mỗi trạng thái "đã dừng" rồi suy ra lỗi mạng, lỗi quyền hay thiếu bộ nhớ là bước nhảy thiếu chứng cứ.

Một dịch vụ cũng có thể còn tiến trình nhưng không còn khả năng làm việc có ích. Vòng lặp bị kẹt, hàng đợi đầy hoặc phụ thuộc phía sau không phản hồi đều có thể khiến PID vẫn tồn tại. Vì vậy, giám sát sự sống của tiến trình và kiểm tra khả năng phục vụ là hai phép đo khác nhau. Ta sẽ gặp lại khác biệt này khi phân tích healthcheck của Docker, probe của Kubernetes và SLI của SRE.

1.6. Áp mô hình vào một sự cố nhỏ

Giả sử sau triển khai, trình duyệt báo không kết nối được tới dịch vụ. Cách suy nghĩ theo lớp bắt đầu từ artifact và tiến trình: bản build đã đến máy chưa; tiến trình có được tạo không; nó còn tồn tại hay đã thoát? Nếu đã thoát, cần xem mã kết thúc và thông báo lỗi. Nếu còn tồn tại, xem nó đã tạo socket lắng nghe đúng địa chỉ và cổng chưa. Sau đó mới xem đường đi mạng và phụ thuộc ở xa. Trật tự này không bảo đảm tìm ra mọi lỗi, nhưng ngăn việc sửa DNS khi chương trình thực ra chưa mở cổng.

Khi kiểm tra, phải phân biệt quan sát với kết luận. “Không thấy PID trong danh sách” là quan sát tại thời điểm đo; “build lỗi” chỉ là một giả thuyết. “Có PID” cũng chưa chứng minh dịch vụ khỏe. Một hồ sơ sự cố tốt ghi mốc thời gian, lệnh hoặc nguồn quan sát, artifact đang chạy, cấu hình có hiệu lực và thay đổi vừa xảy ra. Nhờ thế, người khác có thể phản biện kết luận hoặc tái dựng chuỗi nhân quả thay vì tin vào trí nhớ của người trực hệ thống.

Chương này không yêu cầu thuộc lòng mọi lời gọi hệ thống. Điều cần giữ là ba câu hỏi: thứ gì tồn tại trên đĩa, tiến trình nào đang chạy, và tiến trình ấy đang nắm những tài nguyên nào? Phần sau sẽ dùng cùng ba câu hỏi khi đi từ một tiến trình trên máy tới container, rồi tới workload trong cụm.

Bài tập

C1.1. Một bản triển khai thay tệp thực thi trên đĩa nhưng không khởi động lại dịch vụ. Giải thích vì sao không được kết luận tiến trình đang phục vụ đã chạy phiên bản mới. Nêu phép kiểm tra cần làm.

C1.2. Một chương trình vẫn ghi vào tệp log đã bị đổi tên; tệp mới cùng tên vẫn rỗng. Hãy dùng quan hệ giữa đường dẫn, descriptor và open file description để giải thích. Đề xuất một cách xử lý an toàn.

C1.3. `fork` thành công, `execve` thất bại. Trong mô hình của chương này, có bao nhiêu tiến trình sau `fork`, và tiến trình nào có thể nhận biết lỗi `execve`? Vì sao không được coi `execve` là thao tác tạo tiến trình?

Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu?

Một yêu cầu HTTP mất hai giây không có nghĩa CPU đã làm việc trong hai giây. Trong khoảng ấy, tiến trình có thể đợi lịch chạy, đợi ổ lưu trữ, đợi mạng hoặc chờ khóa do chính chương trình giữ. Nếu không phân biệt những khoảng thời gian này, ta rất dễ tăng số CPU cho một dịch vụ đang nghẽn ở dữ liệu, hoặc tăng bộ nhớ cho một tiến trình đang bị giới hạn thời gian CPU. Chương này xây dựng một cách phân rã độ trễ từ máy tính trước khi dùng công cụ giám sát.

2.1. Công việc, thời gian chạy và thời gian chờ

CPU chỉ thực hiện lệnh của một luồng tại một thời điểm trên mỗi đơn vị xử lý logic. Khi nhiều luồng cùng sẵn sàng, bộ lập lịch chọn luồng nào được chạy tiếp. Một luồng có thể đang chạy, sẵn sàng nhưng chưa được chọn, hoặc không sẵn sàng vì đang chờ một sự kiện. Trạng thái "chờ" không đồng nhất: chờ dữ liệu mạng khác với chờ khóa, và cả hai khác với việc sẵn sàng nhưng bị giới hạn quota CPU. Những lý do này đòi cách sửa khác nhau.

Hãy xét một ví dụ tính toán, không phải kết quả đo: yêu cầu A hoàn tất sau 100 ms theo đồng hồ, trong đó CPU xử lý 40 ms và các khoảng chờ cộng lại 60 ms. Độ trễ của A là 100 ms, còn CPU time gắn với phần xử lý của A là 40 ms. Nếu chỉ giảm thời gian CPU một nửa mà giữ nguyên mọi khoảng chờ, độ trễ mới tốt nhất trong mô hình đơn giản này là 80 ms. Kết quả đó không tiên đoán hệ thống thực; nó chỉ cho thấy không thể lấy tỷ lệ CPU bận để suy trực tiếp độ trễ của từng yêu cầu.

Thông lượng là số công việc hoàn thành trong một khoảng thời gian; độ trễ là thời gian một công việc mất từ đầu đến cuối. Hai đại lượng liên hệ nhưng không thay thế nhau. Một hệ thống có thể giữ thông lượng bằng cách xếp thêm yêu cầu vào hàng đợi, trong khi người dùng chờ lâu hơn. Vì thế, khi nói "cần thêm máy", phải biết mục tiêu là tăng thông lượng, giảm độ trễ ở một mức tải hay giữ hệ thống ổn định khi có đột biến.

Lập lịch công bằng là một mục tiêu chính sách, không phải lời hứa mỗi tiến trình luôn nhận đúng một phần CPU cố định. Chính sách, độ ưu tiên, số tác vụ sẵn sàng, ràng buộc nhóm và phiên bản nhân đều ảnh hưởng kết quả. Tài liệu Linux mô tả quá trình chuyển từ CFS sang EEVDF ở các nhân mới; cuốn sách này giữ mô hình chung "tác vụ sẵn sàng được chọn theo chính sách" ở phần nền tảng, và chỉ bàn thuật toán cụ thể khi có phiên bản nhân làm mốc.

2.2. Địa chỉ ảo không phải RAM đã dùng

Mỗi tiến trình nhìn thấy một không gian địa chỉ ảo. Chương trình dùng các địa chỉ trong không gian ấy; phần cứng và nhân phối hợp ánh xạ chúng tới trang bộ nhớ vật lý khi cần. Hai tiến trình có thể dùng cùng một giá trị địa chỉ ảo mà không mặc nhiên đọc cùng dữ liệu. Điều này tạo ranh giới bảo vệ và cho phép quản lý bộ nhớ linh hoạt hơn một sơ đồ "biến nằm ở ô RAM cố định".

Một vùng địa chỉ được dành cho tiến trình chưa nhất thiết đã có một trang RAM riêng. Chẳng hạn, vùng ánh xạ ẩn danh có thể được tạo trước, còn trang vật lý được cấp khi truy cập hoặc ghi phù hợp. Khi một địa chỉ chưa có ánh xạ vật lý cần thiết, phần cứng báo page fault để nhân xử lý. Page fault

không tự đồng nghĩa lỗi chương trình: nhiều fault là một phần của cấp phát theo nhu cầu. Lỗi thật xuất hiện khi truy cập không hợp lệ hoặc không thể đáp ứng điều kiện ánh xạ.

Vì vậy, một con số "bộ nhớ ảo" lớn không đủ để kết luận sắp hết RAM. Ngược lại, nhìn một con số bộ nhớ của riêng tiến trình cũng chưa cho biết áp lực bộ nhớ toàn máy: trang có thể được chia sẻ, nằm trong cache, hoặc chịu quy tắc tính của nhóm tài nguyên. Khi chẩn đoán, cần xác định phép đo đang đếm địa chỉ, trang cư trú, trang dùng chung hay bộ nhớ tính cho một cgroup. Nhầm tên đại lượng dẫn tới tăng giới hạn sai chỗ.

2.3. Page cache và lời hứa ghi dữ liệu

Đọc một tệp thường đi qua page cache của Linux. Dữ liệu vừa đọc có thể được giữ trong RAM để lần truy cập sau không phải lấy lại từ thiết bị. Cache vì thế không đơn giản là "RAM bị chiếm vô ích"; nó là một cách đổi bộ nhớ lấy độ trễ I/O thấp hơn. Khi thiếu bộ nhớ, nhân có thể thu hồi những trang phù hợp. Tuy nhiên, khả năng thu hồi và chi phí thu hồi khác nhau giữa trang sạch của tệp, trang đã sửa nhưng chưa ghi xuống thiết bị, và vùng nhớ ẩn danh.

Ghi vào tệp cũng có thể hoàn tất ở tầng lời gọi ứng dụng trước khi mọi thay đổi đã nằm trên thiết bị bền vững. Trang đã sửa trong cache được đánh dấu dirty và sau đó cần được đồng bộ. Vì vậy, "lời gọi ghi trả về" và "dữ liệu sống sót qua mất điện" là hai hợp đồng khác nhau. Trên Linux, `fsync` yêu cầu đồng bộ dữ liệu cùng metadata của tệp tới thiết bị theo hợp đồng của lời gọi; khi tạo hoặc đổi tên tệp, việc đồng bộ riêng descriptor của tệp chưa nhất thiết bảo đảm mục thư mục chứa tên đó đã bền vững. Ứng dụng có dữ liệu quan trọng phải xác định chính xác mình cần bảo đảm điều gì, thay vì gọi chung là "đã lưu".

Một thao tác sao lưu an toàn vì thế không chỉ là sao chép bytes rồi thấy lệnh trả 0. Cần kiểm tra nhất quán dữ liệu đầu vào, kết quả ghi, ranh giới bền vững cần thiết và khả năng phục hồi. Hệ thống tệp, cơ sở dữ liệu và object storage cung cấp các hợp đồng khác nhau; chương về lưu trữ sẽ đi sâu vào từng loại. Ở đây điều cần nhớ là I/O có nhiều tầng và độ trễ ở từng tầng không hiện ra từ một phép đo CPU.

2.4. Giới hạn tài nguyên và hành vi dưới tải

Trong môi trường container, cgroup là một cơ chế Linux dùng để tổ chức và kiểm soát tài nguyên của nhóm tác vụ. Một giới hạn CPU có thể khiến nhóm bị trì hoãn khi dùng hết quota trong chu kỳ; một giới hạn bộ nhớ có thể làm tăng thu hồi và cuối cùng dẫn tới lỗi cấp phát hoặc tiến trình bị kết thúc theo chính sách. Hai tình huống này có thể cùng biểu hiện thành tăng độ trễ, nhưng dấu vết và cách xử lý khác nhau. Không nên gọi mọi trường hợp "container chậm" là thiếu CPU.

Để phân biệt, hãy đặt câu hỏi theo chuỗi. Công việc có sẵn sàng chạy nhưng không nhận CPU không? Bộ nhớ nào đang tăng, và đó là cache có thể thu hồi hay dữ liệu hoạt động? Có thao tác I/O nào bị chặn? Giới hạn nào áp cho tiến trình, container hoặc cả máy? Nếu quan sát cho thấy CPU bị giới hạn theo quota, tăng số replica có thể chia tải, nhưng cũng có thể làm áp lực lên phụ thuộc chung. Nếu quan sát cho thấy ứng dụng chờ cơ sở dữ liệu, thêm CPU cho nó không loại bỏ thời gian

chờ ấy.

Những câu hỏi trên là mô hình phân tích, không phải một runbook tự động. Chúng chuẩn bị cho các chương sau về đo lường, HPA và SLO. Trước khi dùng một metric để tự thay đổi số bản sao, phải biết metric đại diện cho tài nguyên nào, cửa sổ thời gian nào và điều gì xảy ra khi phép đo bị thiếu. Một bộ điều khiển ra quyết định từ tín hiệu sai vẫn có thể hoạt động đúng theo thuật toán của nó nhưng tạo kết quả vận hành xấu.

2.5. Lần theo một phút tăng độ trễ

Xét một dịch vụ đọc tệp trong một cgroup v2. Bảng sau là dữ liệu giả định để tập tính và phản biện, không phải đầu ra đo từ máy hay bằng chứng cho một sự cố có thật. Hai cửa sổ đều dài 60 giây, có cùng số yêu cầu và cùng giới hạn tài nguyên. Các giá trị `usage_usec`, `throttled_usec` và `io.pressure some total` là mức tăng của counter tích lũy giữa đầu và cuối mỗi cửa sổ; `memory.current` là giá trị ở cuối cửa sổ. Người đo phải lấy đúng cgroup chứa workload, cùng ranh giới thời gian với số liệu yêu cầu, và kiểm tra các tệp này có sẵn trên nhân đang chạy.

Quan sát trong 60 giây	Bình thường	Độ trễ tăng
p95 yêu cầu	120 ms	870 ms
`cpu.stat usage_usec` tăng	18.000.000 µs	19.200.000 µs
`cpu.stat nr_throttled` tăng	0	0
`memory.current` cuối kỳ	310 MiB	315 MiB
`memory.events high / oom` tăng	0 / 0	0 / 0
`io.pressure some total` tăng	1.200.000 µs	18.000.000 µs

Chia CPU time cho 60 giây cho ta mức dùng trung bình 0,30 rồi 0,32 lõi, không phải 30% rồi 32% độ trễ của từng yêu cầu. `nr_throttled` không tăng nên trong hai cửa sổ này chưa có dấu hiệu cgroup đó bị giới hạn bởi quota CPU; việc luồng phải đợi lịch chạy vì tác vụ khác vẫn có thể xảy ra. Bộ nhớ cuối kỳ tăng 5 MiB và không có sự kiện `high` hay `oom`, nên không có dấu vết vượt hai ngưỡng ấy trong counter đã xem; điều đó không loại trừ thu hồi bộ nhớ toàn máy, page fault hoặc một ngưỡng ở cgroup tổ tiên. Muốn xét áp lực bộ nhớ, đọc thêm `memory.stat`, `memory.pressure` và giới hạn thực tế theo cùng cửa sổ.

Với I/O, `some total` tăng 1,2 giây rồi 18 giây: lần lượt khoảng 2% và 30% của mỗi cửa sổ có ít nhất một tác vụ trong nhóm bị nghẽn I/O. Đây là dấu vết mạnh hơn cho giả thuyết chờ I/O trong cửa sổ chậm, nhưng nó không nói yêu cầu nào chờ, chờ thiết bị nào, hay I/O là nguyên nhân duy nhất của p95 tăng. Bước kế tiếp là ghép thời điểm yêu cầu chậm với thời gian thao tác đọc, bộ đếm bytes/thao tác và tình trạng thiết bị hoặc phụ thuộc tương ứng; cũng phải xét khóa trong ứng dụng và mạng. Nếu đó là đọc từ page cache, số bytes ở thiết bị có thể không phản ánh số lần ứng dụng đọc. Kết luận đúng ở giai đoạn này là ưu tiên kiểm tra đường I/O, chưa phải "ổ đĩa hỏng" hoặc "thêm CPU sẽ hết chậm".

Bài tập

C2.1. Trong một ví dụ giả định, độ trễ yêu cầu là 100 ms gồm 40 ms CPU và 60 ms chờ. Nếu chỉ giảm phần CPU xuống 20 ms, độ trễ thấp nhất theo mô hình cộng đơn giản là bao nhiêu? Nêu một lý do khiến kết quả thực tế khác phép tính.

C2.2. Một tiến trình có không gian địa chỉ ảo lớn nhưng bộ nhớ cư trú thấp. Vì sao chưa thể kết luận nó đang chiếm gần hết RAM? Cần phân biệt ít nhất hai đại lượng nào?

C2.3. Dịch vụ ghi một tệp cấu hình mới rồi đổi tên tệp để thay bản cũ. Việc lời gọi ghi trả về thành công đã đủ chứng minh tên tệp mới sống sót sau mất điện chưa? Giải thích ranh giới cần kiểm tra.

C2.4. Dùng số liệu giả định ở mục 2.5: tính mức dùng CPU trung bình và tỷ phần thời gian `io.pressure some` của hai cửa sổ; nêu dấu vết cho hoặc chống lại ba giả thuyết quota CPU, giới hạn bộ nhớ và chờ I/O. Chỉ ra một điều bảng chưa thể kết luận và phép đo kế tiếp.

Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd

Một dịch vụ có thể chạy dưới đúng người dùng mà vẫn không đọc được cấu hình. Một tệp có bit đọc nhưng đường dẫn tới nó lại không thể đi qua. Một lệnh `systemctl start` có thể trả thành công trong khi ứng dụng chưa sẵn sàng nhận yêu cầu. Những hiện tượng này thuộc các lớp khác nhau: nhân kiểm tra lời gọi hệ thống và quyền truy cập, hệ thống tệp ánh xạ tên sang đối tượng, còn systemd điều hành vòng đời tiến trình. Chương này nối các lớp đó qua một trường hợp vận hành có thể tự kiểm chứng.

3.1. Từ mã người dùng đến lời gọi hệ thống

Tiến trình thực thi phần lớn mã của nó ở user space. Khi cần mở tệp, tạo socket hay đổi quyền, nó gọi một giao diện mà cuối cùng đi vào nhân. Ví dụ `openat` nhận đường dẫn, cờ mở và ngữ cảnh của tiến trình; nhân phân giải đường dẫn, kiểm tra quyền, rồi trả về file descriptor hoặc lỗi. Một hàm thư viện hay một lệnh shell có thể gọi thêm nhiều bước trước và sau lời gọi này. Vì vậy, thông báo “Permission denied” của ứng dụng cần được lần tới thao tác thực sự thất bại; nó không tự cho biết bit quyền nào sai.

Nhân là nơi thực thi các quy tắc về tiến trình, bộ nhớ, đối tượng tệp và thiết bị. systemd là một tiến trình ở user space dùng các giao diện ấy để khởi động và giám sát dịch vụ. `systemctl` lại là công cụ yêu cầu systemd thực hiện thao tác. Ba tên này không thay thế nhau: nhân không đọc mục `Restart=` trong unit, còn systemd không bỏ qua được phép kiểm tra của nhân chỉ vì unit đã nạp thành công.

Sau một lời gọi thất bại, chương trình có thể nhận `EACCES` khi thiếu quyền tìm kiếm hoặc truy cập, `ENOENT` khi thành phần đường dẫn không tồn tại, hay `ENOTDIR` khi một thành phần trung gian không phải thư mục. Đó là mã lỗi ở ranh giới lời gọi; ứng dụng có thể gộp hoặc đổi thông báo khi hiển thị. Trước khi sửa quyền, phải xác định đường dẫn thực, người dùng hiệu lực và thao tác đọc, ghi hay thực thi mà tiến trình đã yêu cầu.

3.2. Tên tệp, inode và bước đi qua thư mục

Đường dẫn là chuỗi các thành phần được tra cứu từ một điểm bắt đầu. Mỗi thư mục tham gia phép tra cứu tên; tệp cuối cùng có metadata như loại, quyền và số inode. Tên không đồng nhất với inode: đổi tên thay quan hệ trong thư mục, còn một descriptor đang mở vẫn tham chiếu đối tượng đã mở như Chương 1 đã giải thích. Một hard link khác có thể trỏ tới cùng inode trên cùng hệ thống tệp, còn symbolic link chứa một đường dẫn cần được phân giải tiếp. Vì thế, xem riêng `ls -l` của tệp đích có thể bỏ sót vấn đề ở thư mục cha hoặc một liên kết trung gian.

Với chế độ quyền Linux thông thường, mỗi nhóm bit đọc, ghi, thực thi áp cho chủ sở hữu, nhóm và người khác. Trên tệp thường, bit `x` cho phép thực thi theo các điều kiện của định dạng tệp; trên thư mục, bit `x` là quyền tìm kiếm/đi qua thành phần tên, không phải “chạy thư mục”. Bit `r` của thư mục cho phép liệt kê tên; biết sẵn một tên và tra cứu nó cần quyền tìm kiếm. Khi tạo hoặc xóa tên trong

thư mục, quyền ở thư mục có vai trò riêng, không thể suy chỉ từ bit ghi trên tệp. ACL, mount option, namespace, chính sách bảo mật bổ sung và quyền đặc biệt có thể làm kết quả khác mô hình ba bộ bit cơ bản; mô hình đó là điểm bắt đầu để khoanh vùng, không phải toàn bộ chính sách.

Xét cây `/srv/atlas/config/app.toml`: `app.toml` là `0640`, thuộc `root:atlas`; tiến trình chạy với nhóm `atlas`. Tệp có vẻ đọc được qua bit nhóm, nhưng nếu `/srv/atlas` là `0700 root:root`, tiến trình không thể đi qua thư mục này và thao tác mở có thể trả `EACCES`. Cách sửa đúng phụ thuộc hợp đồng truy cập: có thể cấp quyền tìm kiếm tối thiểu trên thư mục cho nhóm phù hợp, hoặc đặt cấu hình ở một cây do dịch vụ được phép đi qua. Đổi `app.toml` thành `0666` không sửa được đường đi, lại mở rộng người có thể đọc và sửa bí mật trong tệp. Không dùng `chmod -R 777` như cách chẩn đoán.

Để kiểm chứng trên máy Linux thử nghiệm do mình kiểm soát, ghi nhận danh tính tiến trình bằng `id`, xem từng thành phần bằng `namei -l /srv/atlas/config/app.toml`, rồi thử đọc dưới đúng tài khoản dịch vụ, chẳng hạn `sudo -u atlas cat /srv/atlas/config/app.toml`. Trước khi thay quyền, xác nhận các đường dẫn và tài khoản thực sự tồn tại; các tên trong ví dụ là giả định. `namei` giúp nhìn chuỗi quyền nhưng kết quả đọc cuối cùng mới kiểm tra đúng tổ hợp tài khoản, nhóm, ACL và mount đang có hiệu lực. Nếu unit cấp thêm sandbox hoặc thay root directory, phải kiểm tra từ ngữ cảnh của dịch vụ thay vì chỉ dùng một shell bên ngoài.

3.3. Một unit biến tiến trình thành dịch vụ như thế nào

Một unit service mô tả chương trình chính, danh tính chạy, thứ tự phụ thuộc và chính sách khi nó kết thúc. Ví dụ minh họa sau dùng đường dẫn giả định; trước khi cài phải tạo tài khoản, triển khai chương trình và cấu hình đúng trên máy đích. `Type=exec` khiến thao tác khởi động chờ tới khi thực thi được binary; nó bắt được trường hợp executable hoặc user không tồn tại tốt hơn `Type=simple`. Nó vẫn không chứng minh mã ứng dụng đã đọc xong cấu hình, mở socket hoặc sẵn sàng phục vụ.

```
[Unit]
Description=Atlas example service
After=network.target

[Service]
Type=exec
User=atlas
Group=atlas
WorkingDirectory=/opt/atlas
ExecStart=/opt/atlas/bin/atlas --config /srv/atlas/config/app.toml
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

Dòng `After=network.target` trong unit chỉ đặt thứ tự nếu cả hai unit cùng được đưa vào giao dịch khởi động; nó không tự kéo `network.target` vào, cũng không xác nhận máy đã có địa chỉ IP. `Wants=` kéo một unit khác vào với quan hệ phụ thuộc yếu; `Requires=` mạnh hơn và có thể làm khởi động thất bại khi unit được yêu cầu thất bại, nhưng cả hai không tự đặt thứ tự nếu thiếu

`After=`. `network.target` là mốc thụ động của ngăn xếp mạng, không phải tín hiệu mạng đã cấu hình đầy đủ. Dịch vụ thật sự cần mạng được cấu hình trước khi bắt đầu có thể dùng `Wants=network-online.target` cùng `After=network-online.target`; ý nghĩa "online" do dịch vụ quản lý mạng xác định và vẫn không bảo đảm máy ở xa hay ứng dụng của nó sẵn sàng. Với dịch vụ chạy lâu dài, còn phải thiết kế cách chịu mất mạng và kết nối lại sau khởi động, thay vì coi thứ tự boot là phép kiểm tra sức khỏe.

Trong ví dụ, systemd tạo tiến trình với `User=atlas` và `Group=atlas`, đặt thư mục làm việc ở `/opt/atlas`, rồi thực thi chương trình được chỉ ra ở `ExecStart`. Thư mục làm việc tách khỏi cây cấu hình để lỗi quyền ở `/srv/atlas` xuất hiện khi ứng dụng gọi `openat`, sau bước khởi chạy. Khi ấy nhân vẫn kiểm tra đường dẫn và quyền theo ngữ cảnh đã thiết lập. `Restart=on-failure` yêu cầu thử chạy lại khi tiến trình kết thúc theo điều kiện lỗi được systemd định nghĩa; nó không chữa lỗi quyền cố định. `RestartSec=5s` chỉ tạo khoảng chờ giữa các lần thử và giới hạn tần suất khởi động vẫn có thể chặn vòng lặp lỗi.

Đường lỗi có thể lần theo như sau. Nếu `/opt/atlas/bin/atlas` không tồn tại, bước thực thi thất bại và `Type=exec` báo lỗi khởi động. Nếu binary chạy được nhưng không đi qua `/srv/atlas`, chương trình nhận lỗi khi mở cấu hình và tự thoát với mã lỗi; systemd ghi nhận tiến trình chính đã thoát, áp chính sách restart, rồi có thể chạm giới hạn khởi động. Nếu binary còn chạy nhưng kẹt ở phụ thuộc khác, trạng thái `active` chỉ nói tiến trình được quản lý còn ở trạng thái ấy; nó không phải healthcheck nghiệp vụ. Mỗi đường lỗi cần xem `systemctl status atlas.service`, `journalctl -u atlas.service` và phép thử khả năng phục vụ của chính ứng dụng. Không lấy riêng trạng thái `active` để tuyên bố đã triển khai thành công.

Khi yêu cầu dừng, systemd gửi tín hiệu theo cấu hình dừng và chờ tiến trình kết thúc trong thời hạn quy định; nếu tiến trình không dừng, chính sách có thể dẫn tới tín hiệu cưỡng bức. Ứng dụng phải xử lý tín hiệu phù hợp, ngừng nhận công việc mới, hoàn tất hoặc hủy công việc đang làm theo hợp đồng, đóng tài nguyên và trả mã kết thúc rõ ràng. `Restart=` mô tả phản ứng sau khi tiến trình mất; nó không bảo đảm dữ liệu được ghi bền vững hoặc yêu cầu đang xử lý được giữ lại. Muốn có ranh giới an toàn, phải thiết kế riêng giao thức dừng và kiểm tra nó dưới tải.

3.4. Kiểm chứng trước và sau khi khởi động

Unit có thể được kiểm tra cú pháp và một phần tham chiếu bằng `systemd-analyze verify` trên tệp unit. Sau khi cài unit vào nơi systemd đọc và chạy `daemon-reload`, cần kiểm tra user/group và đường dẫn đã tồn tại, rồi mới khởi động. Trên máy đang quản lý dịch vụ, quan sát `systemctl show` các thuộc tính như `MainPID`, `ExecMainStatus`, `ActiveState` và `SubState` giúp phân biệt "chưa tạo được process", "process thoát" và "process còn sống". Nhật ký đặt các bước vào trục thời gian; cùng một mã thoát chỉ là dấu vết, không phải lời giải thích nguyên nhân.

Một kiểm tra nhỏ nhưng có sức phân biệt là chủ động gây lỗi trong môi trường thử nghiệm: tạm trỏ unit thử tới một binary không tồn tại để quan sát lỗi ở bước exec, rồi trả lại; tiếp đó thử một đường dẫn cấu hình mà tài khoản dịch vụ không đi qua được để quan sát lỗi sau khi process đã khởi chạy.

Hai phép thử không nên làm trên dịch vụ production. Chúng dạy cách đọc ranh giới: trường hợp đầu thuộc thiết lập tiến trình của systemd, trường hợp sau là lời gọi tệp trong ứng dụng bị nhân từ chối. Sau mỗi thay đổi phải hoàn nguyên cấu hình thử, kiểm tra trạng thái và dữ liệu; không dùng restart lặp vô hạn làm bằng chứng tự phục hồi.

Ở điểm này, một dịch vụ có bốn câu hỏi kiểm chứng khác nhau: artifact có ở đúng đường dẫn không; tiến trình có được tạo dưới đúng danh tính không; các lời gọi tài nguyên của nó có thành công không; và người dùng có nhận được chức năng mong đợi không. Chúng lần lượt nối Chương 1, Chương 2 và chương này. Phần mạng sẽ bổ sung đường đi của yêu cầu qua địa chỉ và giao thức, nơi một PID khỏe vẫn có thể không phục vụ được người dùng ở xa.

Bài tập

C3.1. Tệp cấu hình có mode `0640 root:atlas`, tiến trình thuộc nhóm `atlas`, nhưng thư mục cha `/srv/atlas` có mode `0700 root:root`. Vì sao thao tác đọc có thể bị từ chối? Nêu một thay đổi tối thiểu cần kiểm tra và một thay đổi không nên dùng.

C3.2. Với `Type=exec`, `systemctl start` thành công. Điều này xác nhận được bước nào và chưa xác nhận được bước nào của một HTTP service? Nêu hai quan sát tiếp theo.

C3.3. Phân biệt hai lỗi: binary trong `ExecStart` không tồn tại và binary chạy được nhưng `openat` cấu hình trả `EACCES`. Thành phần nào ghi nhận lỗi đầu tiên, systemd có thể phản ứng ra sao, và vì sao tăng `RestartSec` không giải quyết nguyên nhân?

Chương 4. Địa chỉ, định tuyến và TCP: một kết nối đi qua đâu?

Một client báo "không kết nối được" chưa chỉ ra lỗi nằm ở đâu. Nó có thể chọn sai đích, không tìm thấy đường ra, không chuyển được gói tới gateway, bị lọc trên đường, hoặc đến đúng máy nhưng không có chương trình lắng nghe ở địa chỉ và cổng ấy. Muốn sửa đúng, ta phải tách địa chỉ IP của đích, địa chỉ ở từng chặng truyền, trạng thái của kết nối TCP và trạng thái mà Linux đang giữ. Chương này dùng IPv4 và Ethernet làm mô hình cụ thể; IPv6 có cơ chế tìm láng giềng và biểu diễn địa chỉ khác, không được suy nguyên xi từ ARP của IPv4.

4.1. Địa chỉ IP và prefix xác định phạm vi nào?

Địa chỉ IP là định danh logic dùng để chuyển gói qua nhiều mạng. Nó không phải địa chỉ vật lý của card mạng, cũng không phải tên dịch vụ. Một giao diện có thể có nhiều địa chỉ; một máy có thể có nhiều giao diện; cùng một dịch vụ có thể được gọi bằng tên nhưng cuối cùng client vẫn phải chọn một địa chỉ và cổng để thử kết nối. Phần DNS và HTTP ở chương sau mới giải thích đầy đủ bước từ tên tới địa chỉ. Ở đây ta bắt đầu từ đích đã biết là `198.51.100.20:443`.

Giả sử client mang `10.20.1.10/24`. Prefix `/24` nghĩa là 24 bit đầu thuộc phần mạng; trong ví dụ, `10.20.1.10` và `10.20.1.1` cùng prefix `10.20.1.0/24`, còn `198.51.100.20` thì không. Phép tính này chỉ so phần bit theo mask; nó không tự chứng minh hai máy thực sự cùng một miền truyền Ethernet, cũng không cho biết đích có hoạt động. Những địa chỉ `198.51.100.20` và `203.0.113.5` trong chương là địa chỉ dành cho tài liệu, không phải endpoint để độc giả thử trên Internet. Địa chỉ `10.20.1.10` thuộc phạm vi riêng trong ví dụ, được dùng để minh họa một client phía trong gateway.

Linux giữ địa chỉ gán cho giao diện, prefix, trạng thái liên kết và bảng route theo network namespace. `ip address` cho biết cấu hình địa chỉ mà máy đang thấy; nó không chứng minh địa chỉ đã được quảng bá đúng ra mạng hoặc thiết bị phía kia có thể trả lời. Trước khi kết luận "mất mạng", cần kiểm tra client đang chạy trong namespace nào: một container có thể nhìn thấy giao diện và route khác host. Tên giao diện cũng là chi tiết của máy, không phải một hằng số của giao thức.

4.2. Route chọn chặng kế tiếp; ARP tìm địa chỉ trên chặng đó

Với đích ngoài prefix trực tiếp, client tra bảng định tuyến để chọn đường ra và next hop. Trong ví dụ, route mặc định dẫn tới gateway `10.20.1.1` trên giao diện nội bộ. Khi có nhiều route khớp, prefix cụ thể hơn thắng prefix ngắn hơn; route mặc định `/0` chỉ dùng nếu không có đường phù hợp hơn theo các quy tắc chọn route có hiệu lực. Trên Linux, policy routing, nguồn được chọn, bảng route và namespace còn có thể thay đổi kết quả. `ip route get 198.51.100.20` hữu ích vì cho thấy quyết định route đối với một đích cụ thể, gồm gateway và địa chỉ nguồn được chọn trong ngữ cảnh gọi lệnh. Đó là phép xem quyết định ở máy, chưa phải bằng chứng gói đã tới gateway.

Sau khi chọn next hop `10.20.1.1`, client cần địa chỉ lớp liên kết của gateway để gửi frame Ethernet. ARP của IPv4 hỏi "ai giữ địa chỉ IP này?" trên liên kết cục bộ rồi lưu ánh xạ IP–MAC trong bảng láng giềng. Client hỏi MAC của gateway, không hỏi MAC của `198.51.100.20` ở mạng xa. `ip neigh` cho thấy cache láng giềng và trạng thái như `REACHABLE`, `STALE` hoặc `FAILED`; `STALE` không tự nghĩa là mất kết nối vì kernel có thể xác minh lại khi dùng. Một ánh xạ tốt chỉ chứng minh một phần đường cục bộ, không xác nhận route, NAT hay server phía sau.

Router nhận frame ở chặng đầu, bỏ lớp đóng gói liên kết cục bộ, xem địa chỉ IP đích, chọn chặng tiếp theo rồi đóng gói thành frame mới phù hợp với liên kết kế. IP đích thường vẫn là `198.51.100.20` qua các router thông thường, trong khi MAC nguồn và MAC đích đổi theo từng chặng. Nếu router cần chuyển qua nhiều mạng, mỗi chặng tự phân giải láng giềng của nó. Đường trả lời cũng cần route hợp lệ theo chiều ngược; việc client gửi được SYN không có nghĩa SYN-ACK sẽ quay về được.

4.3. TCP tạo một cuộc hội thoại, không tạo đường đi IP

TCP đặt cổng và trạng thái lên trên IP để hai đầu phân biệt các cuộc hội thoại. Với kết nối thường, client gửi SYN từ một cổng nguồn tạm thời tới cổng server, server trả SYN-ACK, rồi client gửi ACK. Các số thứ tự và xác nhận giúp hai đầu đồng bộ trạng thái và nhận biết byte nào đã được chấp nhận. Server phải có socket lắng nghe trên địa chỉ/cổng phù hợp; một tiến trình còn sống hoặc một unit `active` không đồng nghĩa đã có socket lắng nghe ở địa chỉ client đang gọi. `ss -ltn` trên đúng network namespace cho thấy socket TCP đang `LISTEN` và địa chỉ local mà socket bind; `127.0.0.1:443` chỉ nhận kết nối đi vào loopback của chính namespace đó, không nhận gói đến `198.51.100.20:443` qua giao diện ngoài.

Có thể lần theo một trace tối giản với số khởi đầu giả định. Client gửi `SYN seq=1000`; server đáp `SYN-ACK seq=5000 ack=1001`; client gửi `ACK seq=1001 ack=5001`. Mỗi SYN chiếm một vị trí trong không gian số thứ tự dù không mang dữ liệu. Sau đó client gửi `ACK seq=1001 ack=5001 len=4`, tức các byte được đánh số 1001–1004; server đáp `ACK seq=5001 ack=1005 len=0`, nghĩa là đã nhận liên tục mọi byte trước 1005 và đang chờ byte 1005. Nếu riêng ACK này mất, client có thể truyền lại chính bốn byte với `seq=1001 ack=5001 len=4`; server nhận ra dữ liệu lặp, không giao lần hai cho ứng dụng và lại báo `ack=1005`. ACK là cộng dồn, không phải dấu xác nhận riêng từng packet. Trace bỏ qua TCP option, dữ liệu trong SYN, đảo thứ tự, vòng số thứ tự và cách bộ hẹn giờ quyết định truyền lại; capture ở một điểm cũng không đủ chứng minh vì sao bản truyền lại xuất hiện.

Một kết nối được nhận diện bởi giao thức và bộ địa chỉ, cổng ở hai đầu. `SYN-SENT` ở client cho biết nỗ lực bắt tay chưa hoàn tất; `ESTABLISHED` cho biết hai TCP endpoint đã thiết lập trạng thái, chưa chứng minh HTTP, TLS hay ứng dụng trả kết quả đúng. Nếu cổng đích không có người lắng nghe, client có thể nhận reset và `connect` báo `ECONNREFUSED`. Nếu SYN hoặc phản hồi không trở lại, TCP có thể thử lại và cuối cùng báo hết thời gian; `ETIMEDOUT` chỉ là triệu chứng, không tự chỉ ra firewall, mất route hay server quá tải. Một firewall `REJECT` cũng có thể tạo phản hồi rõ, còn `DROP` thường làm client chờ; cần xem gói thực tế và chính sách có hiệu lực trước khi quy nguyên nhân.

Sau bắt tay, TCP không chỉ "gửi byte rồi hy vọng". Bên gửi giữ dữ liệu chưa được xác nhận để truyền lại khi phát hiện mất; ACK xác nhận tiến độ của luồng byte. Cửa sổ nhận giới hạn lượng dữ liệu bên nhận còn nhận được, còn điều khiển nghẽn giới hạn lượng dữ liệu bên gửi đưa vào mạng theo tín hiệu của đường truyền. Hai giới hạn khác nhau: cửa sổ nhận nhỏ có thể phản ánh ứng dụng đọc chậm, còn điều khiển nghẽn phản ứng với tình trạng mạng theo thuật toán đang dùng. Việc bắt tay thành công không bảo đảm tốc độ truyền tiếp theo. TCP đóng có trạng thái riêng; reset chấm dứt đột ngột, còn đóng thông thường trao đổi FIN và có thể để lại `TIME-WAIT` nhằm xử lý các segment cũ. Không coi mọi socket không còn `ESTABLISHED` là lỗi.

4.4. NAT đổi thông tin trên đường và cần đường về

Để client dùng địa chỉ riêng ra mạng khác, gateway trong ví dụ thực hiện NAPT: nó đổi nguồn `10.20.1.10:51514` thành `203.0.113.5:40001` khi gửi tới `198.51.100.20:443`, đồng thời giữ ánh xạ để dịch gói trả lời về đúng client. Số cổng `40001` chỉ là giá trị giả định; triển khai có thể giữ hoặc đổi cổng theo khả năng và xung đột. Server thấy nguồn sau dịch, không thấy địa chỉ riêng của client ở tầng IP. Nếu không có ánh xạ trả về còn hiệu lực, hoặc gói đáp đi qua một thiết bị khác không biết ánh xạ, phản hồi có thể thất lạc dù SYN ban đầu đã rời client.

NAT không phải định tuyến: route quyết định chặng chuyển tiếp, còn NAT thay địa chỉ hoặc cổng trong gói ở điểm dịch và cập nhật checksum cần thiết. Trạng thái dịch theo luồng của gateway cũng khác trạng thái TCP trong hai endpoint. Linux có thể dùng hệ thống theo dõi kết nối của netfilter để gắn các gói với cùng luồng và áp dụng NAT, nhưng việc thấy một mục theo dõi không tự chứng minh server đã chấp nhận kết nối. Ngược lại, một capture sau NAT có tuple khác capture trước NAT; phải ghi rõ mình quan sát ở giao diện nào và phía nào của điểm dịch trước khi so hai trace. NAT không phải cơ chế bảo mật thay cho lọc gói và chính sách truy cập.

4.5. Lần theo một client không kết nối được

Đây là tình huống giả định có kết quả quan sát được định nghĩa trước để tập suy luận, không phải log của một hệ thống thật. Client `10.20.1.10/24` cần kết nối `198.51.100.20:443` qua gateway `10.20.1.1`; gateway dịch nguồn sang `203.0.113.5`. Ứng dụng báo `ECONNREFUSED`. Mục tiêu không phải thử mọi lệnh, mà tìm điểm đầu tiên nơi trạng thái quan sát khác đường đi mong đợi.

Trước hết, xác định lời gọi kết nối dùng đúng IP và cổng, rồi xem route trong namespace của client. Giả sử `ip route get` chọn `10.20.1.1` và địa chỉ nguồn `10.20.1.10`; `ip neigh` cho gateway có ánh xạ hợp lệ. Hai quan sát loại bớt giả thuyết thiếu route hoặc chưa tìm được MAC ở client tại thời điểm đo, nhưng không chứng minh gateway đang chuyển tiếp. Một capture ngắn ở giao diện ra của client thấy SYN `10.20.1.10:51514` tới `198.51.100.20:443`; như vậy lỗi không nằm ở việc client chưa phát SYN. Nếu không có quyền quan sát gateway, phải nói rõ khoảng đường giữa hai điểm đo vẫn chưa xác minh.

Trong tình huống này, người vận hành có quyền xem gateway và server. Capture ở phía ngoài gateway thấy SYN với nguồn đã đổi thành `203.0.113.5:40001`. Capture trên server thấy đúng

SYN tới `198.51.100.20:443`, rồi một RST từ phía server quay lại. Điều đó cho thấy route đi và ánh xạ ra đã làm việc cho gói này; reset là manh mối mạnh cho việc kết nối bị từ chối tại hoặc gần server. Nó chưa tự phân biệt kernel vì không có listener với firewall chủ động từ chối. Ta kiểm tra đúng namespace của server: `ss -ltn` chỉ thấy `127.0.0.1:443`, không thấy listener trên `198.51.100.20:443` hay địa chỉ bao quát thích hợp. Đồng thời kiểm tra quy tắc lọc có hiệu lực để loại trừ reset do chính sách. Với bộ quan sát giả định này, nguyên nhân trực tiếp là ứng dụng chỉ bind vào loopback; sửa cấu hình bind theo hợp đồng truy cập rồi xác nhận `ss` và bắt tay SYN, SYN-ACK, ACK trên đúng đường. Sau đó mới thử một yêu cầu ứng dụng: bắt tay TCP thành công chưa kiểm tra được TLS hoặc HTTP.

Nếu trace khác đi, kết luận cũng đổi. Không có SYN tại client dẫn ta về địa chỉ đích, route hoặc lỗi cục bộ; SYN rời client nhưng không tới gateway dẫn ta về link, neighbor hoặc lọc tại chặng đầu. SYN tới server mà không có phản hồi cần kiểm tra lọc, trạng thái máy và việc server nhận gói; SYN-ACK rời server nhưng không về client hướng chú ý sang route trả, NAT và lọc theo chiều về. Những nhánh này là giả thuyết có thứ tự, không phải bảng ánh xạ một triệu chứng với một nguyên nhân duy nhất. Capture là ảnh chụp ở một điểm và một thời gian: mất gói khi capture, offload, namespace sai hoặc lưu lượng qua đường khác đều có thể làm nó thiếu bức tranh. Bằng chứng tốt nhất ghép cùng tuple kết nối, thời gian và hai đầu của đoạn đường đang nghi vấn.

Chương này dừng ở việc TCP đưa được luồng byte giữa hai endpoint. Người dùng còn có thể báo "không vào được trang" vì tên miền trỏ sai, chứng chỉ bị từ chối, reverse proxy chọn sai upstream hoặc ứng dụng trả lỗi. Các bước ấy thuộc tầng tiếp theo. Giữ ranh giới giúp ta không sửa route khi TCP đã bắt tay, và không sửa HTTP khi SYN chưa tới server.

Bài tập

C4.1. Client `10.20.1.10/24` gửi tới `198.51.100.20`. Route mặc định qua `10.20.1.1`. Client cần tìm MAC của địa chỉ nào trước khi gửi frame đầu tiên? Vì sao không hỏi trực tiếp MAC của server?

C4.2. Một máy có route mặc định và route `198.51.100.0/24` qua hai gateway khác nhau. Với đích `198.51.100.20`, route nào được ưu tiên? Nêu một lý do khiến kết quả thực tế trên Linux vẫn cần kiểm tra bằng `ip route get` trong đúng namespace.

C4.3. Client thấy SYN rời máy; server thấy SYN và gửi SYN-ACK; client không thấy SYN-ACK. Nêu ít nhất hai điểm cần kiểm tra theo chiều về và giải thích vì sao chưa thể quy lỗi cho ứng dụng HTTP.

C4.4. Gateway đổi `10.20.1.10:51514` thành `203.0.113.5:40001`. Server trả gói về tuple nào, và gateway cần điều kiện gì để đưa gói về client? Phân biệt trạng thái NAT với trạng thái TCP endpoint.

C4.5. `connect` trả `ECONNREFUSED`; server chỉ có `127.0.0.1:443` ở trạng thái `LISTEN`. Hãy nêu giả thuyết mạnh nhất, một nguyên nhân cạnh tranh vẫn phải kiểm tra, và phép xác nhận sau khi sửa.

Vì sao `systemctl active` chưa đủ?

C4.6. Trong trace mục 4.3, vì sao SYN của client được đáp bằng `ack=1001`, còn bốn byte ở `seq=1001` được đáp bằng `ack=1005`? Nếu ACK đầu cho dữ liệu bị mất, client truyền lại cùng bốn byte thì server nên báo số ACK nào, và vì sao ứng dụng không nhận dữ liệu hai lần?

Chương 5. Từ tên miền đến phản hồi: DNS, TLS, HTTP và reverse proxy

Một trang không mở được có thể bắt đầu từ tên miền sai, nhưng cũng có thể thất bại sau khi client đã đến đúng IP và hoàn tất TCP. Ta sẽ theo một yêu cầu giả định tới `https://api.example.test/orders/42` qua một reverse proxy rồi tới ứng dụng phía sau. Đường chính của chương là DNS chọn địa chỉ, TCP nối tới địa chỉ ấy, TLS xác thực và bảo vệ kênh client–proxy, HTTP mang yêu cầu, proxy chọn upstream, rồi phản hồi đi ngược về. Đây là mô hình HTTPS trên TCP với HTTP/1.1 và TLS 1.3 để thấy rõ từng ranh giới; Không nên dùng mô hình này để khẳng định mọi lần tải trang đều mở một kết nối mới hoặc đi đủ mọi bước khi cache và kết nối tái sử dụng còn hiệu lực.

5.1. DNS trả dữ liệu tên; client vẫn phải chọn đích

Tên `api.example.test` không tự mang địa chỉ IP. Ứng dụng nhờ bộ phân giải ở máy client tìm bản ghi phù hợp, thường là A cho IPv4 hoặc AAAA cho IPv6. Bộ phân giải có thể dùng thông tin cục bộ hoặc hỏi recursive resolver; resolver này dùng cache hay hỏi các máy chủ có thẩm quyền theo hệ phân cấp. Kết quả DNS là dữ liệu để client chọn địa chỉ thử kết nối, không phải phép thử cổng 443, chứng chỉ, hay tình trạng ứng dụng. Cùng một tên có thể trả nhiều địa chỉ; client và mạng có thể chọn các đường khác nhau. Khi đo, cần ghi rõ loại bản ghi, resolver, thời điểm và địa chỉ thật mà tiến trình đã dùng, thay vì coi một kết quả tra cứu trên máy khác là bằng chứng của request lỗi.

Mỗi bản ghi có TTL giới hạn thời gian lưu ở cache DNS. Thay bản ghi ở authoritative server không buộc mọi cache và mọi tiến trình chuyển ngay sang địa chỉ mới; một số ứng dụng còn có cache hoặc giữ kết nối của riêng mình. Kết quả không tồn tại tên (NXDOMAIN) cũng có thể được cache theo quy tắc riêng; nó khác việc resolver chờ quá hạn vì không nhận được câu trả lời đáng tin. Vì thế "DNS sai" cần nói sai ở đâu: dữ liệu authoritative, cache của resolver, cấu hình resolver ở client, hay tên mà ứng dụng thật sự hỏi. Giảm TTL trước một thay đổi có kế hoạch có thể rút ngắn thời gian giữ dữ liệu cũ, đổi lại tăng số lần tra cứu; nó không xóa được cache đã nhận TTL trước đó. DNS thành công chỉ mở cửa cho bước kết nối tiếp theo.

5.2. TCP và TLS giải quyết hai câu hỏi khác nhau

Với địa chỉ đã chọn, client dùng route và TCP của Chương 4 để nối tới cổng 443 của proxy. Không có đường trả về hoặc SYN bị lọc có thể gây timeout trước khi có bất kỳ HTTP status nào. TCP `ESTABLISHED` chỉ chứng minh luồng byte đã mở giữa client và đầu nhận TCP; nó chưa nói đầu kia là đúng tên dịch vụ. Một phép thử trực tiếp bằng IP có thể bỏ qua DNS, nhưng nếu không giữ nguyên tên dịch vụ cho TLS và HTTP thì phép thử đó có thể đi vào cấu hình virtual host khác.

Trên TCP, client bắt đầu TLS bằng ClientHello, thường mang tên đích qua SNI để máy phục vụ chọn cấu hình và chứng chỉ phù hợp. Trong TLS 1.3 với xác thực bằng chứng chỉ, hai bên thỏa thuận khóa; server gửi chứng chỉ và chứng minh nắm khóa tương ứng, còn các thông điệp hoàn tất kiểm tra tính toàn vẹn của bắt tay. Client phải kiểm tra chuỗi tin cậy, thời hạn và danh tính trong chứng chỉ có

khớp tên dịch vụ yêu cầu. Nếu tên không khớp hoặc chuỗi không đáng tin, client phải dừng theo chính sách xác thực của mình; tắt xác minh chỉ che lỗi và mở đường cho kẻ khác mạo danh. SNI giúp chọn chứng chỉ, không phải chứng cứ rằng chứng chỉ đã hợp lệ. Tên trong HTTP `Host` được dùng ở tầng định tuyến yêu cầu; nó cũng không thay thế phép xác minh TLS.

Khi TLS kết thúc ở reverse proxy, chỉ kênh client–proxy được bảo vệ bởi phiên TLS ấy. Proxy có thể mở một kết nối HTTP thường hoặc một phiên TLS khác tới upstream; cần xác định hợp đồng mạng và mối đe dọa của đoạn sau trước khi chọn. TLS tới proxy không tự mã hóa đoạn proxy–upstream. TLS handshake thành công cũng chưa chứng minh proxy chọn đúng ứng dụng hay ứng dụng xử lý được yêu cầu. Tái sử dụng kết nối và phiên có thể giảm chi phí bắt tay, nhưng làm cho chẩn đoán "một request bằng một kết nối mới" sai; khi kiểm tra cần biết client vừa mở hay đang dùng kết nối cũ.

5.3. HTTP mang ý định; proxy quyết định chuyển tiếp

HTTP/2 có thể ghép nhiều luồng trên một kết nối TCP; HTTP/3 vận chuyển cùng ngữ nghĩa HTTP trên QUIC. Vì vậy khi chẩn đoán cần biết giao thức đã thương lượng, thay vì giả định một request luôn ứng với một kết nối TCP mới.

Sau TLS, client gửi một HTTP request có phương thức, đích yêu cầu, các trường như `Host` và có thể có body. Với ví dụ này, `GET /orders/42` yêu cầu một tài nguyên; phản hồi có status, header và có thể có nội dung. Status mô tả kết quả ở tầng HTTP, nhưng một `200` không chứng minh nội dung đúng, còn một `404` có thể do proxy hoặc ứng dụng trả. Nếu HTTP request chưa hề được gửi vì DNS, TCP hoặc TLS hỏng thì không có status HTTP từ đích để diễn giải. Trong HTTP/1.1, `Host` giúp chọn authority khi nhiều tên cùng dùng một IP; một TCP connection tới đúng IP vẫn có thể bị route sang virtual host khác nếu tên yêu cầu sai.

Reverse proxy nhận request ở phía client, chọn route theo tên và đường dẫn, rồi gửi một request sang upstream đã cấu hình. Đây là hai cuộc trao đổi có thể dùng hai kết nối và hai bộ timeout khác nhau. Proxy có thể thêm thông tin về client ban đầu cho upstream, nhưng ứng dụng chỉ nên tin các trường chuyển tiếp do proxy đáng tin cậy đặt hoặc chuẩn hóa; client ngoài cũng có thể tự gửi header cùng tên. Proxy còn có thể cân bằng tải, giới hạn truy cập, đổi header hoặc cache theo cấu hình. Mỗi tính năng tạo thêm điểm cần quan sát: proxy trả lời từ cache thì upstream có thể không thấy request; upstream trả nhanh nhưng proxy gửi chậm thì độ trễ người dùng vẫn cao. Log nên ghép thời điểm, tên đích, route đã chọn, upstream, thời gian kết nối, thời gian chờ phản hồi và một định danh request không chứa bí mật.

Trace dưới đây là dữ liệu giả định để đọc một lần trao đổi thành công, không phải output thu từ mạng. Các dòng phân cách chặng là chú giải, dòng trống kết thúc phần header; byte trên dây HTTP/1.1 dùng CRLF. Client xác thực TLS với `api.example.test` rồi gửi request bên trong kênh mã hóa tới proxy. Proxy giải mã ở điểm kết thúc TLS và, theo cấu hình giả định này, mở HTTP thường tới upstream; đoạn sau cần biện pháp bảo vệ riêng nếu đi qua miền mạng không tin cậy.

```
client -> proxy (HTTP inside TLS)
GET /orders/42 HTTP/1.1
Host: api.example.test
Accept: application/json

proxy -> upstream (HTTP, illustrative)
GET /orders/42 HTTP/1.1
Host: orders.internal.test
X-Forwarded-Host: api.example.test
X-Forwarded-Proto: https

upstream -> proxy
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 9

{"id":42}

proxy -> client (HTTP inside TLS)
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 9

{"id":42}
```

`Host: api.example.test` giúp proxy chọn virtual host công khai. Ở chặng sau, proxy chủ động đổi `Host` thành `orders.internal.test` để upstream định tuyến, đồng thời ghi lại tên và giao thức phía client bằng hai trường chuyển tiếp do chính proxy ghi đè; upstream chỉ nên tin chúng khi request đến từ proxy được tin cậy. `200 OK` trong trace do upstream tạo và proxy chuyển tiếp nguyên trạng, không phải bằng chứng proxy luôn giữ nguyên header hoặc body. `Content-Length: 9` đếm chín byte ASCII của `{"id":42}`. Nếu upstream không trả header kịp ngưỡng chờ, proxy có thể tự sinh `504` thay cho phản hồi này; khi đó cần log của cả hai chặng để biết request có tới upstream và có còn chạy sau lúc proxy ngừng chờ hay không. Trace bỏ qua kết nối tái sử dụng, cache và cách đóng kết nối để tập trung vào ranh giới trách nhiệm.

Nếu proxy, trong vai gateway, nhận phản hồi không hợp lệ từ upstream, nó có thể tạo `502 Bad Gateway`; nếu không nhận phản hồi kịp để hoàn tất yêu cầu, nó có thể tạo `504 Gateway Timeout`. Đây là nghĩa của status, không phải bảng kết luận nguyên nhân: một upstream hoặc tầng trung gian cũng có thể tự trả mã giống vậy, và log proxy phải cho biết mã nào do nó tạo. Lỗi nối upstream, TLS tới upstream, header sai và ứng dụng đóng kết nối là những giả thuyết khác nhau cần phân biệt bằng log và phép đo ở đúng đoạn. Một timeout ngắn giảm thời gian chờ nhưng có thể loại yêu cầu hợp lệ chậm; timeout dài giữ tài nguyên và kéo dài trải nghiệm lỗi. Thử upstream khác có thể cải thiện khả dụng, nhưng gửi lại một yêu cầu thay đổi trạng thái khi chưa biết lần đầu đã thực thi chưa có thể tạo tác dụng phụ lặp. Chính sách retry phải dựa vào tính idempotent, khóa chống lặp hoặc hợp đồng ứng dụng, không chỉ dựa vào status.

Cache HTTP lưu phản hồi đủ điều kiện và dùng độ tươi hoặc xác minh lại để giảm độ trễ và tải upstream. Nó khác cache DNS: một bên lưu dữ liệu tên, bên kia lưu phản hồi ứng dụng. Cấu hình cache thiếu phân biệt người dùng có thể làm lộ nội dung riêng; việc thấy nội dung cũ cũng chưa đủ

kết luận DNS còn cũ. Cần kiểm tra `Cache-Control`, khóa cache, `Age` và đường đi thực tế theo loại phản hồi. Không dùng cache để che một upstream hỏng nếu dữ liệu sai hoặc quá cũ có hậu quả nghiệp vụ.

5.4. Một sự cố, nhiều ranh giới cần đo

Xét một mô hình giả định để luyện chẩn đoán, không phải trace từ hệ thống thật. Theo cấu hình mong đợi, `api.example.test` trỏ tới IP của proxy mới; proxy nhận HTTPS và chuyển `/orders/42` tới upstream đang phục vụ. Người dùng báo "không mở được đơn hàng" sau thay đổi cấu hình. Trước khi sửa, người vận hành ghi lại URL, thời điểm, client, lỗi hiển thị và request ID nếu có. Cùng một câu báo lỗi có thể tương ứng năm điểm dừng sau đây; mỗi phép quan sát chỉ chứng minh một đoạn đường.

Nếu máy bị lỗi phân giải tên ra IP của proxy cũ, còn authoritative server và một resolver khác đã trả IP mới, giả thuyết đầu là cache hoặc cấu hình resolver trên đường của máy ấy. Hỏi đúng resolver mà client dùng, xem TTL còn lại và địa chỉ tiến trình thực sự kết nối; không sửa upstream của proxy mới khi request chưa đến đó. Nếu DNS trả đúng IP mới nhưng SYN rời client mà không có SYN-ACK, chuyển sang route, lọc và đường trả TCP như Chương 4. Một DNS answer đúng không chứng minh cổng mở; timeout TCP chưa phải lỗi TLS hay HTTP. Quan sát ở proxy xem SYN có đến hay không giúp thu hẹp đoạn lỗi, nhưng một capture im lặng tại sai giao diện không đủ kết luận gói chưa tới máy.

Nếu TCP hoàn tất rồi client báo certificate error, xem tên mà client yêu cầu, SNI, chứng chỉ proxy đã chọn, chuỗi tin cậy và đồng hồ của máy. Trong nhánh giả định này, proxy mới đưa chứng chỉ chỉ chứa `old.example.test`, nên phép xác minh tên `api.example.test` thất bại; sửa binding/chứng chỉ đúng tên là hành động phù hợp. Không chuyển sang HTTP hay bỏ xác minh chứng chỉ để "thử cho qua". Khi TLS hợp lệ và client nhận `502`, đối chiếu log theo request ID: proxy chọn upstream nào, có nối được không, có nhận header HTTP hợp lệ không, hay upstream tự trả `502`. Giả sử log cho thấy upstream chấp nhận TCP rồi đóng trước khi gửi header, nguyên nhân nằm ở giao tiếp proxy–upstream hoặc ứng dụng phía sau, chưa thể suy riêng từ mã `502` là tiến trình đã chết.

Cuối cùng, nếu proxy nhận request nhưng chờ upstream quá hạn rồi tự trả `504`, xem thời gian ở từng bước và log của cùng request ở upstream. Giả sử upstream ghi nhận request và kẹt ở truy vấn dữ liệu lâu hơn ngưỡng chờ của proxy: việc nâng timeout có thể giảm `504` trước mắt nhưng không sửa truy vấn chậm, đồng thời giữ tài nguyên lâu hơn. Cũng phải kiểm tra upstream có thể đã hoàn tất tác dụng phụ sau khi proxy bỏ chờ; retry tự động một lệnh ghi có thể tạo đơn hàng trùng. Mỗi nhánh cần một phép xác nhận sau sửa từ đúng đường người dùng: tên phân giải đúng, TCP bắt tay, TLS xác thực, proxy chọn đúng upstream và response có nội dung đúng. Status, log hay capture riêng lẻ chỉ là một phần chứng cứ.

Bài tập

C5.1. Client A vẫn lấy IP proxy cũ cho `api.example.test`, trong khi một resolver khác trả IP mới. Nêu ba dữ kiện cần ghi trước khi quy lỗi cho authoritative DNS. Vì sao một lần tra cứu từ máy của người vận hành không đủ?

C5.2. TCP tới proxy đã `ESTABLISHED` nhưng client báo chứng chỉ không khớp tên. Nêu thông tin cần đối chiếu ở client và proxy. Vì sao đổi HTTP `Host` hoặc tắt xác minh TLS không phải cách sửa gốc?

C5.3. Cùng URL lần lượt gặp timeout trước TLS, certificate error, `502`, `504`. Với mỗi dấu hiệu, hãy chỉ ra ranh giới xa nhất chắc chắn đã vượt qua và một quan sát tiếp theo có giá trị phân biệt. Có thể kết luận upstream chết chỉ từ `502` không?

C5.4. Proxy hết thời gian chờ một yêu cầu tạo đơn hàng rồi định retry sang upstream khác. Nêu rủi ro và điều kiện để retry an toàn. Phân biệt tác dụng của tăng timeout với sửa nguyên nhân upstream chậm.

Chương 6. Git là đồ thị lịch sử, không phải thư mục sao lưu

Khi hai người cùng sửa một dịch vụ, câu hỏi không chỉ là "tệp cuối cùng trông ra sao" mà còn là thay đổi nào đã được ghi, thay đổi nào chưa, và hai dòng phát triển có chung tổ tiên nào. Git lưu các snapshot có liên kết cha–con. Hiểu liên kết ấy giúp ta chọn cách tích hợp, đọc conflict và phục hồi sự cố mà không làm mất lịch sử của đồng đội. Các ký tự `A`, `B`, `C` trong chương là nhãn tượng trưng cho commit, không phải SHA thật; tình huống là mô hình giả định, không phải log của repo này.

6.1. Từ nội dung tệp tới commit có cha

Một blob chứa nội dung tệp, không chứa tên tệp. Một tree ghép tên, mode và định danh object của blob hoặc tree con thành snapshot thư mục. Commit trỏ tới một tree gốc, ghi tác giả, người tạo commit, thông điệp và không hoặc nhiều commit cha. Vì vậy đổi tên tệp có thể đổi tree mà blob nội dung vẫn giữ nguyên; hai commit khác nhau có thể trỏ tới cùng snapshot tệp nhưng khác cha hoặc metadata. Git lưu lịch sử bằng các quan hệ này, không phải bằng cách chép nguyên một thư mục có nhãn thời gian cho mỗi commit.

Định danh object là giá trị băm của loại, độ dài và nội dung object theo định dạng kho đang dùng. Nhiều kho dùng SHA-1; Git cũng hỗ trợ kho SHA-256. Vì commit chứa định danh tree và cha, thay nội dung một tệp làm đổi blob, tree chứa nó và các commit mới đi qua thay đổi ấy. Định danh cho phép kiểm tra tính nhất quán nội dung trong kho; nó không tự chứng minh ai là tác giả hay rằng mã an toàn. Tên branch và tên file không nằm trong blob. Để tự đối chiếu mô hình trên một kho được phép đọc, `git cat-file -p HEAD` cho thấy trường tree và parent của commit; `git ls-tree HEAD` cho thấy các mục ở tree gốc. Một SHA rút gọn chỉ là cách viết đủ phân biệt trong kho hiện tại, không phải một loại object khác.

6.2. Ba trạng thái cần phân biệt trước khi commit

Working tree là những tệp đang thấy và sửa bằng trình biên tập. Index là snapshot dự kiến cho commit kế tiếp; `git add` chép nội dung hiện tại của đường dẫn vào index tại thời điểm chạy. `HEAD` thường là tham chiếu tượng trưng tới branch đang checkout, còn branch ref trỏ tới một commit. Khi tạo commit, Git ghi tree từ index, đặt commit mới có cha là commit tại `HEAD`, rồi dịch ref của branch hiện tại sang commit mới. Sửa tệp sau `git add` không tự cập nhật index; commit có thể chứa bản cũ trong khi working tree vẫn có sửa đổi mới.

Giả sử `timeout_ms` trong commit hiện tại là `1000`. Ta sửa working tree thành `2000`, stage, rồi sửa tiếp thành `3000`. Lúc ấy `HEAD` giữ `1000`, index giữ `2000`, working tree giữ `3000`. `git diff --cached` so `HEAD` với index, còn `git diff` mặc định so index với working tree. Nếu commit ngay, snapshot mới ghi `2000`; muốn ghi `3000` phải stage lại và kiểm tra diff. Đây là lý do `git status` và diff trước commit có giá trị hơn lời khẳng định "tôi đã lưu file". Tệp chưa được theo dõi chưa tự có trong index.

6.3. Branch là ref; merge là kết quả của hai dòng lịch sử

Xét commit `B` có `timeout_ms=1000`. Nhánh `main` đi từ `B` tới `D`, đặt `timeout_ms=3000` để giảm timeout sai ở cổng vào. Nhánh `feature/retry` đi từ `B` tới `C`, đặt cùng dòng thành `timeout_ms=2000` để thử retry. Đồ thị trước tích hợp là `A → B → D` trên `main` và `B → C` trên `feature/retry`; `B` là tổ tiên chung gần nhất. Tên branch chỉ là ref có thể dịch chuyển; `D` và `C` là hai object commit vẫn còn nguyên khi ref đổi.

Nếu `main` vẫn ở `B`, tích hợp `C` có thể fast-forward: chỉ dịch ref `main` tới `C`, không tạo commit hai cha. Ở đây `main` đã tới `D`, nên fast-forward không đủ. Merge ba phía so snapshot ở `B`, `D` và `C`. Cả hai phía thay cùng dòng theo hai cách nên Git không thể chọn đúng ý định nghiệp vụ; nó dừng với conflict. Trước khi người vận hành giải quyết, `HEAD` vẫn là `D`, `MERGE_HEAD` ghi phía `C`, index có thể giữ ba stage cho tệp xung đột: stage 1 là bản ở `B`, stage 2 là phía `HEAD`/`D`, stage 3 là phía `MERGE_HEAD`/`C`. Dấu `<<<<<<<`, `=======`, `>>>>>>>` trong working tree chỉ là trình bày vùng chưa giải quyết, không phải cấu hình được phép đưa vào production.

Quyết định đúng không thể suy từ câu "lấy ours" hay "lấy theirs". Nhóm phải kiểm tra hợp đồng timeout và retry. Giả sử kiểm thử cho thấy upstream cần ngưỡng `2500` và retry chỉ dành cho thao tác idempotent. Người tích hợp viết `timeout_ms=2500`, cập nhật kiểm thử liên quan, rồi stage tệp đã giải quyết. Khi đó index có bản thường thay cho ba stage xung đột; `git diff --cached` cho thấy snapshot sắp ghi, và `git ls-files -u` không còn mục chưa giải quyết. Chạy kiểm thử cấu hình và hành vi retry trên kết quả cuối, xem diff để bảo đảm không còn marker; chỉ sau đó tạo merge commit `M` với hai cha theo thứ tự `D` và `C`. Ref `main` dịch tới `M`, còn ref `feature/retry` vẫn ở `C` nếu chưa xóa. Nếu chưa hiểu cách giải, `git merge --abort` có thể đưa merge về trước khi bắt đầu, nhưng với thay đổi chưa commit trước merge không nên mặc định rằng mọi thứ luôn khôi phục hoàn hảo; hãy bắt đầu từ working tree sạch.

6.4. Rebase thay gốc bằng commit mới

Thay vì merge, trên nhánh feature chưa chia sẻ có thể rebase `C` lên `D`: Git lấy thay đổi mà `C` đưa vào so với `B`, thử áp dụng lên `D`, rồi tạo commit mới `C'` có cha là `D`. Vì `D` đã sửa cùng dòng, rebase cũng có thể dừng để con người giải conflict. Sau khi giải và tiếp tục, ref feature trỏ tới `C'`; `C` cũ không bị sửa tại chỗ. Ngay cả khi snapshot cuối cùng giống kết quả merge, đồ thị khác: `A → B → D → C'` là một đường, trong khi merge tạo `M` có hai cha `D` và `C`. Giá trị băm của `C'` khác `C` vì nội dung commit, đặc biệt parent, khác.

Rebase thuận tiện để sắp xếp các commit của chính mình trước khi công bố. Nếu người khác đã dựa vào `C` trên một branch dùng chung, đẩy lịch sử rebase lên sẽ buộc ref từ xa rời đường cũ và làm bản sao của họ lệch nhau; cần phối hợp rõ trước mọi lần viết lại lịch sử đã chia sẻ. Với nhánh tình huống này đã có người cùng dùng, chọn merge để giữ cả `C` và `D` trong lịch sử công khai. Cả merge lẫn rebase chỉ xử lý cách ghép lịch sử và nội dung; không tự chứng minh bản `timeout_ms=2500` đúng. Chạy lại phép kiểm chứng trên snapshot tích hợp là điều bắt buộc.

6.5. Revert thêm lịch sử; reset dịch vị trí

Nếu một commit thường đã công bố gây lỗi, `git revert` tạo commit mới đảo tác động của commit được chọn trên trạng thái hiện tại. Commit lỗi vẫn có trong đồ thị, người khác có thể lấy commit đảo chiều bằng cách đồng bộ bình thường; revert cũng có thể xung đột nếu mã đã thay đổi thêm. Muốn revert chính merge commit `M` của ví dụ, phải chỉ định parent làm dòng chính: `git revert -m 1 M` xem `D` là mainline rồi đảo thay đổi mà merge đưa vào so với `D`. Cần xem diff và kiểm thử vì thao tác này không có nghĩa "quay toàn bộ repo về B", và về sau merge lại cùng nhánh có thể không tự mang lại các thay đổi đã bị đảo.

`git reset <commit>` ở dạng dịch `HEAD` đặt ref branch hiện tại về commit được chọn; chế độ `--soft`, mặc định `--mixed`, và `--hard` lần lượt ảnh hưởng khác nhau tới index và working tree. `--hard` còn có thể bỏ thay đổi local chưa commit ở các tệp được theo dõi. Dịch ref `main` từ `M` về `D` sẽ làm lịch sử branch công bố không còn đi qua `M`, dù object có thể còn tạm được tham chiếu ở nơi khác. Vì thế reset là công cụ sửa lịch sử cục bộ khi đã hiểu trạng thái; nó không thay revert trên nhánh đã chia sẻ. Khôi phục một tệp trong working tree hoặc bỏ stage là tác vụ khác với dịch branch ref; đừng dùng một lệnh dịch lịch sử để chữa nhầm một file đang sửa.

6.6. Đọc chứng cứ trước khi chọn hành động

Khi nhận một conflict thật, trước hết xác định branch đang checkout, working tree có sạch trước khi tích hợp không, và hai commit đầu nhánh cùng merge base. Sau đó đọc ba phiên bản của tệp, xác định mỗi thay đổi phục vụ yêu cầu nào; conflict marker chỉ cho vị trí chồng lấn, không cho quyết định đúng. Kiểm tra diff đã stage và đồ thị cha sau commit, chạy test trên snapshot vừa tích hợp, rồi mới chia sẻ. Nếu test thất bại, phân biệt lỗi từ code đã commit với sửa đổi chỉ còn trong working tree; một `git status` sạch không thay thế kiểm thử. Nếu cần hủy ảnh hưởng trên branch chung, chuẩn bị một revert có review; nếu cần thử lại một merge đang dở, kiểm tra trạng thái trước khi abort. Sự an toàn đến từ việc biết ref, index và working tree nào đang thay đổi ở mỗi bước.

Bài tập

C6.1. Commit `B` chứa `timeout_ms=1000`. Sau `git add` bản `2000`, người học sửa working tree thành `3000` nhưng chưa stage lại. `HEAD`, index và working tree lần lượt giữ gì? Commit ngay sẽ ghi giá trị nào và phép so sánh nào chứng minh điều đó?

C6.2. Trên đồ thị `A → B → D` của `main` và `B → C` của feature, vì sao không thể fast-forward `main` tới `C` mà giữ `D`? Nếu giải conflict rồi merge thành `M`, nêu hai cha và trạng thái các branch ref. Index có gì khi conflict đang dở?

C6.3. Nếu rebase `C` lên `D` và giải cùng conflict, đồ thị mới và định danh commit feature thay đổi ra sao? Vì sao phương án này cần cân nhắc khác khi `C` đã được chia sẻ? Hai cách tích hợp có tự xác nhận timeout mới là đúng không?

C6.4. Merge commit `M` đã lên branch chung rồi gây lỗi. So sánh mục tiêu và hậu quả của revert `M` với reset `main` về `D`; vì sao revert merge cần chọn mainline? Cần kiểm chứng gì trước khi push?

Chương 7. Bash và ranh giới lỗi trong tự động hóa

Một script backup có thể in `BACKUP_OK` dù không tạo được bản sao dùng được. Lỗi không nhất thiết nằm ở lệnh sao lưu: shell có thể biến dữ liệu thành đối số khác, pipeline có thể che mã lỗi, và thao tác dọn dẹp có thể chạy sau một kết luận thành công sai. Chương này xem Bash như một chương trình điều phối tiến trình và file descriptor. Mọi đường dẫn và kết quả của tình huống bên dưới thuộc một thử nghiệm cô lập; chúng không mô tả trạng thái của lab hoặc máy production.

7.1. Shell biến dòng chữ thành lệnh như thế nào?

Bash đọc cú pháp trước: dấu nháy, toán tử `|`, `>`, `&&` và dấu kết thúc lệnh xác định cấu trúc. Sau đó shell mở rộng các word: biến, phép tính, command substitution, tách word và mở rộng tên tệp theo ngữ cảnh, rồi bỏ dấu nháy. Ký tự đặc biệt sinh ra từ giá trị biến không tự được phân tích lại thành toán tử shell; gọi `eval` để ép phân tích lần hai là mở một ranh giới thực thi mới. Dấu nháy kép giữ kết quả mở rộng biến thành một đối số, trừ những quy tắc riêng như `"$@"` giữ từng đối số ban đầu. Dấu nháy đơn giữ nguyên hầu hết ký tự bên trong, không mở rộng biến.

Giả sử `src='/tmp/du lieu'`. Với `tar -C $src`, Bash có thể đưa cho `tar` hai đối số `/tmp/du` và `lieu`; `tar -C "$src"` đưa đúng một đường dẫn. Nếu biến không được nháy chứa `*`, pathname expansion còn có thể thay một word thành nhiều tên file. Điều này xảy ra trước khi `tar` chạy; sửa tùy chọn của `tar` không chữa được lỗi truyền đối số. Phép gán đơn giản `src=$1` không tách word ở vế phải, nhưng dùng `$src` không nháy ở lời gọi sau đó vẫn nguy hiểm. Với tập tên file tùy ý, dùng mảng Bash và `"${files[@]}"`; không ghép danh sách thành một chuỗi rồi hy vọng shell tách lại đúng. `--` là quy ước được nhiều utility hỗ trợ để kết thúc phần tùy chọn, không phải phép bảo vệ do Bash tự áp cho mọi chương trình.

Một biến chưa đặt và một biến rỗng là hai trạng thái khác nhau. `set -u` báo lỗi khi mở rộng biến chưa đặt trong nhiều ngữ cảnh, nhưng không chứng minh đường dẫn tồn tại hoặc nội dung hợp lệ. Cú pháp `${1:?can tham so}` buộc có đối số thứ nhất và khác rỗng trước khi đi tiếp; nó vẫn không bảo đảm đối số ấy là một thư mục an toàn. Dữ liệu từ môi trường, tham số và file cấu hình phải đi qua kiểm tra theo hợp đồng của thao tác sắp làm.

7.2. Redirection, pipeline và exit status

Mỗi tiến trình nhận các file descriptor chuẩn: 0 để đọc, 1 để ghi thông thường, 2 để ghi lỗi. `cmd >file 2>&1` cho descriptor 1 trỏ tới file rồi nối descriptor 2 vào đích hiện tại của 1. Đảo thứ tự thành `cmd 2>&1 >file` khiến descriptor 2 giữ đích cũ của 1 trong khi 1 mới trỏ tới file. Bash thực hiện redirection trước khi thực thi lệnh; lỗi mở file có thể làm lệnh chính không chạy. Với pipeline, output của lệnh trái trở thành input của lệnh phải, và các phần thường chạy trong những môi trường tiến trình riêng. Đừng kỳ vọng biến gán bên trong một nhánh pipeline luôn trở lại shell cha; cấu hình `lastpipe` là một ngoại lệ có điều kiện.

Mặc định, exit status của `tar ... | tee log` là status của `tee`, không phải của `tar`. Nếu `tar` thoát 2 còn `tee` ghi log thành công và thoát 0, cả pipeline có status 0. `set -o pipefail` đổi status pipeline thành status khác 0 của lệnh lỗi nằm ngoài cùng bên phải; nó không nói lệnh nào hỏng nếu nhiều lệnh hỏng. Mảng `PIPESTATUS` trong Bash giữ status từng phần, nhưng phải sao chép ngay: chỉ cần chạy thêm một lệnh là trạng thái quan sát đã đổi. Status 0 nói rằng chương trình đã hoàn thành theo hợp đồng riêng của nó; không chứng minh bản backup khôi phục được. GNU `tar` phân biệt 0 thành công, 1 khi file thay đổi trong lúc tạo kho và 2 cho lỗi nghiêm trọng; `tee` thành công chỉ nói việc ghi dữ liệu nó nhận đã thành công theo hợp đồng của `tee`.

7.3. Vì sao `set -e` không phải bộ xử lý lỗi?

`set -e` yêu cầu shell thoát khi một lệnh đơn giản thất bại trong những ngữ cảnh nhất định. Nó bỏ qua lỗi ở phép thử `if`/`while`, ở phần không cuối của chuỗi `&&`/`||`, ở lệnh không cuối của pipeline khi không bật `pipefail`, và ở một số ngữ cảnh phủ định. Nếu một hàm được gọi làm điều kiện `if`, các lệnh bên trong cũng chịu ngữ cảnh bỏ qua này: `false` có thể không chặn lệnh sau. Đây là quy tắc điều khiển của Bash, không phải một ngoại lệ của utility. `set -euo pipefail` có ích để bắt một số sơ suất, nhưng không định nghĩa thế nào là dữ liệu hợp lệ hoặc việc khôi phục thành công.

Command substitution `result=$(cmd)` chạy `cmd` trong môi trường subshell và bỏ các newline ở cuối output; status của phép gán đơn giản phản ánh substitution cuối. Bash không ở POSIX mode thường tắt `errexit` trong môi trường substitution, trừ khi bật `inherit_errexit`; vì vậy các lệnh bên trong cần xử lý status rõ. Một bẫy khác là `local result=$(cmd)`: status người gọi thấy có thể là của builtin `local`, che status của `cmd`. Tách `local result` và `result=$(cmd) || ...` khi kết quả quan trọng. Không nhét một thao tác có side effect vào `echo "$(...)"` rồi chỉ kiểm tra status của `echo`.

7.4. Tín hiệu, trap và việc chạy lại

Bash khởi tạo tiến trình con cho utility bên ngoài, chờ status và quyết định nhánh tiếp theo. `trap ... EXIT` chạy khi shell thoát bình thường hoặc do lỗi khiến shell thoát; `trap ... INT` và `TERM` xử lý tín hiệu tương ứng nếu shell có cơ hội nhận và xử lý chúng. `SIGKILL` không bắt được. Khi Bash đang chờ một lệnh đồng bộ ở foreground, trap có thể bị hoãn tới lúc lệnh đó kết thúc. Một trap trên shell cha không tự hủy toàn bộ cây tiến trình con. Vì vậy trap phù hợp để dọn file tạm đã biết, nhưng không thay cơ chế quản lý process group, timeout và shutdown của trình quản lý dịch vụ.

Khi script chạy lại, phải định nghĩa rõ trạng thái nào có thể tái sử dụng. Việc `backup.tar` đã tồn tại không chứng minh nó đầy đủ; việc bản mới được tạo cũng không có nghĩa được phép xóa bản cũ. Một cách an toàn hơn là ghi vào file tạm trong chính thư mục đích, kiểm tra thành công của lệnh tạo, rồi công bố bằng thao tác không ghi đè tên cuối. Điều này xử lý xung đột tên, nhưng không giải quyết nguồn thay đổi trong lúc đọc, mất điện trước khi dữ liệu được đồng bộ, hay đối thủ đổi đường dẫn giữa kiểm tra và sử dụng. Quyền truy cập thư mục, snapshot nhất quán, kiểm thử khôi phục và chính sách giữ bản cũ là những hợp đồng riêng.

7.5. Tình huống: backup báo thành công giả

Ví dụ lỗi dưới đây nhận thư mục nguồn và thư mục đích đã tồn tại. Nó được thiết kế để quan sát lỗi status, không có lệnh xóa nguồn. Không dùng nó cho backup thật.

```
#!/usr/bin/env bash
# Ví dụ lỗi: chỉ chạy trên dữ liệu thử nghiệm.
set -e
src=$1
out=$2
tar -cf "$out/backup.tar" -C "$src" . 2>&1 | tee "$out/backup.log"
printf 'BACKUP_OK\n'
```

Trong thử nghiệm trên Bash 5.2.21, GNU tar 1.35 và GNU coreutils 9.4, nguồn đầu vào là một thư mục không tồn tại, còn thư mục đích là thư mục tạm rỗng. `tar` không thể đổi sang nguồn và thoát 2; `tee` ghi thông báo lỗi vào log rồi thoát 0. Vì không bật `pipefail`, status pipeline bằng 0; `set -e` không chặn `printf`, script in `BACKUP_OK` và thoát 0. Dòng `BACKUP_OK` vì thế là kết luận của shell dựa trên status bị che, không phải chứng cứ backup tốt. Trên hệ thống khác, nội dung và ngôn ngữ thông báo lỗi, cũng như file tar dở dang, có thể khác. Trong một script có bước xóa nguồn phụ thuộc status này, hậu quả có thể là mất dữ liệu; ví dụ thực thi ở đây cố ý không có bước ấy.

Phiên bản sau giữ từng đường lỗi ở vị trí có thể kiểm tra. Nó yêu cầu GNU `mktemp`, `tar`, `ln`, `rm`; tên cuối cố định và không cho ghi đè. `mktemp` tạo file riêng trong thư mục đích để tránh va chạm tên file tạm. Mọi nhánh trước `ln` đều thoát khác 0 và `EXIT` trap chỉ dọn đúng đường dẫn tạm đã tạo. `tar -tf` kiểm tra archive đọc được về mặt cấu trúc, không chứng minh dữ liệu có thể phục hồi đúng nghiệp vụ. `ln -T` tạo hard link tới file tạm trên cùng filesystem và buộc đối số đích được hiểu là đúng một tên file, kể cả khi tên ấy là symlink tới thư mục; nếu `backup.tar` đã tồn tại, GNU `ln` mặc định thất bại, nên lần chạy lại không âm thầm thay bản trước. Thành công của `ln` là điểm công bố tên, sau đó trap gỡ tên tạm và để hard link cuối giữ inode. Đích phải là thư mục tin cậy; một cây đường dẫn bị thay đổi đồng thời hoặc do đối thủ kiểm soát cần thiết kế an toàn hơn.

```
#!/usr/bin/env bash
# GNU tar/coreutils; đích phải là thư mục tin cậy cùng filesystem.
set -u
src=${1:?can thu muc nguon}
out=${2:?can thu muc dich}
[[ -d $src && -d $out ]] || exit 2

tmp=$(mktemp -- "$out/.backup.XXXXXX") || exit 3
cleanup() { rm -f -- "$tmp"; }
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM

if tar -cf "$tmp" -C "$src" .; then
    :
else
    exit $?
fi
if tar -tf "$tmp" >/dev/null; then
    :
else
    exit 4
fi
ln -T -- "$tmp" "$out/backup.tar" || exit 5
printf 'BACKUP_OK\n'
```

Ví dụ kiểm chứng cô lập tạo một file `payload.txt` trong thư mục nguồn tạm, chạy script sửa lần đầu, rồi liệt kê archive và trích xuất sang một thư mục tạm khác để so nội dung. Lần thứ hai trả status 5 vì tên cuối đã tồn tại; bản cuối của lần đầu vẫn đọc được. Khi thay nguồn bằng đường dẫn không tồn tại, script dừng status 2 trước khi tạo file tạm. Những kết quả này kiểm tra đường thành công, lỗi đầu vào và xung đột tên cho môi trường thử; chúng không chứng minh tính bền vững sau mất điện, tính nhất quán của dữ liệu đang bị ghi, hay an toàn trước thay đổi đường dẫn có chủ ý. Một backup production cần snapshot hoặc điểm dừng ghi phù hợp ứng dụng, checksum/manifest, thử khôi phục định kỳ, chính sách retention và đồng bộ tệp/thư mục theo hệ thống tệp thực tế. Bài học nối với Chương 2 về I/O bền vững, Chương 3 về đường dẫn/quyền và Chương 6 về khác biệt giữa một snapshot đã ghi với working tree đang đổi.

Bài tập

C7.1. Cho `src='/tmp/du lieu'`, giải thích số đối số đường dẫn mà `tar -C $src` và `tar -C "$src"` nhận sau mở rộng. Nếu tên file chứa *, vì sao chỉ thêm `set -u` không sửa lỗi? Nêu cách giữ từng phần tử của một danh sách tên file.

C7.2. Trong script lỗi, giả sử `tar` trả 2 và `tee` trả 0. Dự đoán status pipeline, việc in `BACKUP_OK` và status cuối của script khi chưa bật `pipefail`. Bật `pipefail` đổi điều gì? Status 0 của `tar` sẽ và sẽ không chứng minh điều gì?

C7.3. Xét `f() { false; printf 'done\n'; }` rồi `if f; then printf 'ok\n'; fi` với `set -e`. Dự đoán các dòng in ra theo Bash và giải thích vì sao `false` không tự làm script thoát. Viết lại ý định “dừng nếu bước đầu thất bại” mà không dựa vào `set -e`.

C7.4. Trong script sửa, sau một lần thành công, lần chạy lại với cùng đích đi tới bước nào và status nào? Vì sao `tar -tf` cùng việc không ghi đè chưa đủ cho backup production? Phân biệt lỗi được `EXIT` trap dọn với rủi ro do `SIGKILL` hoặc nguồn thay đổi giữa chừng.

Chương 8. Python và Go: điều phối tác vụ, giữ đúng ranh giới lỗi

Chương 7 cho thấy một pipeline Bash có thể che mã lỗi của bước sao lưu. Chuyển cùng việc ấy sang Python hoặc Go không tự sửa sai: chương trình vẫn có thể nhận lỗi rồi bỏ qua và in thông báo thành công. Khác biệt hữu ích nằm ở cách biểu diễn đối số, kết quả, thời hạn và trạng thái của từng bước. Ta sẽ điều phối ba bước nhỏ: kiểm tra điều kiện, áp dụng thay đổi giả, xác minh kết quả. Không bước nào chạm hệ thống thật; helper chỉ ghi vào thư mục tạm được đánh dấu cho demo.

8.1. Khi nào giữ Bash, khi nào dùng Python hoặc Go?

Bash phù hợp cho vài lệnh tuyến tính trên một máy, với đầu vào nhỏ và hợp đồng lỗi rõ. Khi phải giữ trạng thái qua nhiều bước, đọc cấu hình có cấu trúc, kiểm tra dữ liệu, phân loại lỗi và kiểm thử nhánh phục hồi, Python thường cho mã dễ đọc hơn. Thư viện chuẩn của Python đã có subprocess, xử lý tệp, JSON và kiểm thử; đổi ngôn ngữ chỉ để gọi một lệnh đơn lẻ không mang lại an toàn. Go phù hợp khi cần một chương trình biên dịch triển khai trên nhiều máy, kiểu dữ liệu và luồng lỗi tường minh, hoặc nhiều tác vụ I/O đồng thời với cancellation chung. Một binary Go vẫn phụ thuộc hệ điều hành, kiến trúc, tùy chọn build và có thể phụ thuộc thư viện động; "một file" không có nghĩa chạy được ở mọi nơi. Python cần interpreter và môi trường package tương thích. Quyết định dựa trên độ phức tạp trạng thái, môi trường triển khai, năng lực nhóm và chi phí vận hành, không dựa trên nhãn "ngôn ngữ DevOps".

Trong cả ba ngôn ngữ, chương trình điều phối không đồng nhất với tác vụ nó khởi chạy. Một lệnh con có thể chạy xong và trả status khác 0; không khởi chạy được; bị timeout; hoặc trả 0 nhưng kết quả nghiệp vụ sai. Từng trạng thái cần chính sách riêng. Exit status là quan sát từ tiến trình con, còn "triển khai thành công" là kết luận sau khi kiểm tra hậu điều kiện. Không ghép hai ý ấy thành một biến boolean duy nhất.

8.2. Đối số, cấu hình và môi trường thực thi

Python `subprocess.run` nhận danh sách đối số; Go `exec.CommandContext` nhận tên chương trình cùng từng đối số. Hai API này mặc định không gọi shell. Giá trị như `"a; rm -rf ..."` đi thành một đối số, không thành lệnh thứ hai. Điều đó chặn một kiểu shell injection tại ranh giới này, nhưng utility con vẫn có thể hiểu dấu gạch đầu như tùy chọn, đọc file cấu hình độc hại hoặc tự gọi shell. Nếu cố dùng `shell=True` hay `sh -c` với chuỗi ghép từ đầu vào, ranh giới parsing của Chương 7 trở lại. Dùng argv tách rời, xác nhận giá trị theo schema và hợp đồng của chương trình con, chọn đường dẫn executable đáng tin, rồi dùng dấu kết thúc tùy chọn khi utility hỗ trợ.

Đường dẫn làm việc, `PATH`, biến môi trường, locale, quyền và tài nguyên của tiến trình con ảnh hưởng kết quả. Python `env=None` và Go `Cmd.Env=nil` cho con kế thừa môi trường hiện tại; điều này tiện trong demo nhưng có thể chuyển cả credential hoặc cấu hình ngoài dự kiến. Bí mật trong argv có thể lộ qua danh sách tiến trình hoặc log; bí mật trong environment cũng không phải két an toàn tuyệt đối. Đừng in nguyên command, exception, environment hoặc output khi chúng có thể

chứa secret. Chỉ ghi mã bước, loại lỗi và request ID đã lọc; cấp secret qua cơ chế quản lý riêng với quyền tối thiểu. Trong ví dụ, output con bị bỏ đi vì ta đã biết helper nhỏ và chỉ cần status; công cụ thật cần log giới hạn kích thước, che dữ liệu nhạy cảm và đủ chứng cứ chẩn đoán.

8.3. Một lỗi được quan sát qua Python và Go

Helper sau là tác vụ giả. Nó chỉ hoạt động khi thư mục đích có file `.demo-only`. Sau khi khởi tạo, nó phát `READY` rồi chờ lệnh `go`: nhờ vậy hạn tác vụ chỉ bắt đầu khi cả hai phía đã sẵn sàng. Biến `DEMO_STARTUP_DELAY` chỉ dùng trong kiểm thử để mô phỏng máy khởi tạo chậm. `apply_fail` ghi `partial` rồi trả 7. `apply_slow` ghi `partial`, ngủ hai giây, rồi mới định ghi `ready`. `verify` chỉ trả 0 nếu trạng thái là `ready`. Như vậy một lỗi hoặc timeout có thể để lại dấu vết của tác dụng phụ trước khi tiến trình con dừng. Không suy từ timeout rằng không có gì đã xảy ra.

```
"""Demo child: only write inside a marked temporary directory."""
import os
from pathlib import Path
import sys
import time

if len(sys.argv) != 3:
    sys.exit(2)
action, work_arg = sys.argv[1:]
work = Path(work_arg)
if not (work / ".demo-only").is_file():
    sys.exit(2)
state = work / "state.txt"
time.sleep(float(os.environ.get("DEMO_STARTUP_DELAY", "0")))
print("READY", flush=True)
if sys.stdin.readline() != "go\n":
    sys.exit(2)
if action == "precheck":
    sys.exit(0)
if action == "apply_ok":
    state.write_text("ready\n", encoding="utf-8")
    sys.exit(0)
if action in ("apply_fail", "apply_slow"):
    state.write_text("partial\n", encoding="utf-8")
    if action == "apply_fail":
        sys.exit(7)
    time.sleep(2)
    state.write_text("ready\n", encoding="utf-8")
    sys.exit(0)
if action == "verify":
    sys.exit(0 if state.is_file() and
             state.read_text(encoding="utf-8") == "ready\n" else 8)
sys.exit(2)
```

Chương trình Python gọi cùng helper theo thứ tự `precheck → apply → verify`. Với lệnh đơn giản, `subprocess.run(check=False)` trả `CompletedProcess` khi con kết thúc, kể cả status 7 hoặc 8; người gọi phải đọc `returncode`. Ở đây ta dùng `Popen` để đợi `READY` riêng, rồi mới gửi `go` và gọi `communicate(timeout=0.5)`. Quá hạn tác vụ, chương trình kill và reap con trực tiếp; lỗi khởi chạy hoặc mất tín hiệu sẵn sàng được gắn nhãn khác với lỗi bước. Nhãn `TIMEOUT` không phải

exit status của helper. Nhánh `bad` ghi nhận lỗi nhưng vẫn chạy bước sau và in `DEPLOY_OK`; nhánh `checked` dừng khác 0 ngay ở lỗi đầu. Cả hai nhánh cố ý để lại `partial` để người học thấy kết luận sai và giới hạn của việc dừng sớm.

```
"""Demo only: 'bad' deliberately ignores a failed step."""
import queue
import subprocess
import sys
import threading

STARTUP_LIMIT = 10  # Chỉ chờ tín hiệu READY, không tính vào hạn tác vụ.
TASK_LIMIT = 0.5


def run_step(probe, work, action):
    try:
        child = subprocess.Popen(
            [sys.executable, probe, action, work],
            stdin=subprocess.PIPE, stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL,
        )
    except OSError:
        return "START_ERROR"
    ready = queue.Queue(maxsize=1)
    threading.Thread(target=lambda: ready.put(child.stdout.readline()),
                     daemon=True).start()
    try:
        line = ready.get(timeout=STARTUP_LIMIT)
    except queue.Empty:
        child.kill()
        child.wait()
        return "START_TIMEOUT"
    if line.strip() != b"READY":
        child.kill()
        child.wait()
        return "START_ERROR"
    try:
        child.communicate(input=b"go\n", timeout=TASK_LIMIT)
    except subprocess.TimeoutExpired:
        child.kill()
        child.communicate()
        return "TIMEOUT"
    except OSError:
        child.kill()
        child.wait()
        return "IO_ERROR"
```

```
    return "OK" if child.returncode == 0 else f"EXIT {child.returncode}"

if len(sys.argv) != 5:
    sys.exit("usage: script.py bad|checked ok|fail|slow probe.py workdir")
policy, mode, probe, work = sys.argv[1:]
if policy not in ("bad", "checked") or mode not in ("ok", "fail", "slow"):
    sys.exit(2)

for action in ("precheck", "apply_" + mode, "verify"):
    status = run_step(probe, work, action)
    if status != "OK":
        print(f"{action}: {status}", file=sys.stderr)
        if policy == "checked":
            sys.exit(1)
print("DEPLOY_OK")
```

Go dùng `exec.CommandContext` nhưng chưa hủy context trong giai đoạn đợi `READY`. Sau tín hiệu đó, hàm gửi `go`, bắt đầu bộ đếm 0,5 giây và đợi `Wait`; nếu hết hạn, nó hủy context để kill con trực tiếp rồi reap. Với API ngắn hơn, `Run` trả `nil` khi lệnh chạy và kết thúc thành công; ở đây `Wait` có cùng ranh giới lỗi sau `Start`. Status khác 0 thường cho `*exec.ExitError`, còn lỗi tìm executable hoặc I/O là loại lỗi khác. Mã phải kiểm tra kết quả, không chỉ ghi log rồi bỏ qua. Nhánh `bad` dưới đây mắc đúng lỗi quyết định đó; nhánh `checked` dừng. Output con ngoài tín hiệu đồng bộ bị bỏ khỏi log demo, không phải mẫu quan sát phù hợp cho mọi dịch vụ.

```
// Demo only: policy "bad" deliberately ignores a failed step.
package main

import (
 "fmt"
 "os"
)

func main() {
 if len(os.Args) != 6 {
  fmt.Fprintln(os.Stderr, "usage: app bad|checked ok|fail|slow python probe workdir")
  os.Exit(2)
 }
 policy, mode := os.Args[1], os.Args[2]
 if (policy != "bad" && policy != "checked") ||
  (mode != "ok" && mode != "fail" && mode != "slow") {
  os.Exit(2)
 }
 python, probe, work := os.Args[3], os.Args[4], os.Args[5]
 for _, action := range []string{"precheck", "apply_" + mode, "verify"} {
  status := runStep(python, probe, work, action)
  if status != "OK" {
   fmt.Fprintf(os.Stderr, "%s: %s\n", action, status)
   if policy == "checked" {
    os.Exit(1)
   }
  }
 }
 fmt.Println("DEPLOY_OK")
}
```

Hàm chạy từng bước tách rời chính sách `bad/checked`: nó chỉ trả loại kết quả quan sát được. Cờ `TIMEOUT` do bộ đếm của cha xác định; lỗi trả về từ `Wait` sau khi kill không phải exit status nghiệp vụ của helper.

```
package main

import (
 "bufio"
 "context"
 "errors"
 "fmt"
 "io"
 "os/exec"
 "strings"
 "time"
)

func runStep(python, probe, work, action string) string {
 ctx, cancel := context.WithCancel(context.Background())
 defer cancel()
 cmd := exec.CommandContext(ctx, python, probe, action, work)
 input, err := cmd.StdinPipe()
 if err != nil {
  return "START_ERROR"
 }
 output, err := cmd.StdoutPipe()
 if err != nil {
  return "START_ERROR"
 }
 cmd.Stderr = io.Discard
 if err = cmd.Start(); err != nil {
  return "START_ERROR"
 }
 ready := make(chan string, 1)
 go func() {
  line, _ := bufio.NewReader(output).ReadString('\n')
  ready <- line
 }()
 select {
 case line := <-ready:
  if strings.TrimSpace(line) != "READY" {
   cancel()
   cmd.Wait()
   return "START_ERROR"
  }
 case <-time.After(10 * time.Second):
```

```
    cancel()
    cmd.Wait()
    return "START_TIMEOUT"
   }
   if _, err = io.WriteString(input, "go\n"); err != nil {
    cancel()
    cmd.Wait()
    return "IO_ERROR"
   }
   input.Close()
   done := make(chan error, 1)
   go func() { done <- cmd.Wait() }()
   select {
   case err = <-done:
    if err == nil {
     return "OK"
    }
    var exit *exec.ExitError
    if errors.As(err, &exit) {
     return fmt.Sprintf("EXIT %d", exit.ExitCode())
    }
    return "IO_ERROR"
   case <-time.After(500 * time.Millisecond):
    cancel()
    <-done
    return "TIMEOUT"
   }
  }
```

Ma trận kiểm thử có kiểm soát phải đối chiếu cả status lẫn file trạng thái: `ok` in `DEPLOY_OK`, thoát 0 và có `ready`; `bad fail` in `DEPLOY_OK` dù `apply` trả 7, `verify` trả 8 và file còn `partial`; `checked fail` thoát 1 trước `verify`; `checked slow` thoát 1 vì timeout sau khi đã ghi `partial`. `bad slow` vẫn in thành công giả. Ca `checked ok` còn ép khởi tạo helper chậm hơn hạn cũ 0,3 giây để chứng minh đồng hồ tác vụ chưa chạy trong lúc khởi tạo. Đây là phép thử của demo, không phải chứng nhận cho quy trình triển khai thật.

8.4. Timeout, cancellation và thất bại từng phần

Python `run(timeout=...)` bao cả thời gian giao tiếp với con sau khi nó được tạo, nhưng khởi tạo tiến trình trên một số nền tảng có thể chưa bị ngắt ngay đúng deadline. Go `CommandContext` gắn context cho command và mặc định kill process khi context kết thúc; nó không tự rollback file đã ghi. Cả hai ví dụ chỉ quản lý tiến trình con trực tiếp. Nếu con khởi chạy cháu, giữ pipe mở hoặc gọi một API từ xa đã nhận yêu cầu, hết hạn ở phía điều phối không chứng minh mọi tác vụ bên ngoài đã dừng. Với Go, việc chờ pipe đóng và cách hủy có thể cần `WaitDelay` hoặc quy ước shutdown riêng; với Python, một process group có thể cần quản lý riêng. Không mặc định kill toàn bộ cây tiến trình bằng một lời gọi.

Sau lỗi, phải xác định trạng thái bền vững hiện tại rồi mới quyết định thử lại, tiếp tục hay quay lui. Retry một thao tác ghi có thể nhân đôi tác dụng nếu lần đầu đã hoàn tất ở hệ thống đích nhưng phản hồi bị mất, như tình huống HTTP ở Chương 5. Chỉ retry khi thao tác idempotent theo hợp đồng,

hoặc có khóa chống lặp và khả năng truy kết quả cũ; dùng số lần và tổng thời gian hữu hạn. `finally` của Python và `defer` của Go phù hợp giải phóng tài nguyên cục bộ, nhưng không thay giao dịch hay rollback phân tán; `os.Exit` trong Go còn bỏ qua các `defer` đang chờ ở cùng tiến trình. Việc xóa file `partial` mù quáng có thể che trạng thái cần điều tra hoặc xóa công việc hợp lệ của lượt khác. Demo dừng và giữ dấu vết, không tự tuyên bố đã khôi phục.

8.5. Đóng gói và kiểm chứng trước khi chạy ở nơi khác

Hai ví dụ chỉ dùng thư viện chuẩn. Với Python, ghi rõ phiên bản interpreter, tạo virtual environment cho package của dự án và khóa dependency theo công cụ đóng gói đã chọn; virtual environment là thứ tái tạo được, không nên coi thư mục của nó là artifact có thể di chuyển nguyên vẹn giữa máy. Với Go, module ghi yêu cầu ở `go.mod`, checksum dependency được quản lý ở `go.sum`; cần ghi cả phiên bản toolchain, target OS/architecture và cờ build khi muốn tái tạo binary. Không đưa secret vào package hoặc Git. Test nên chạy trong thư mục tạm có dấu hiệu nhận diện, xác minh cả status lẫn trạng thái cuối, rồi xóa đúng thư mục ấy sau khi đối chiếu. Với thay đổi thật, thêm kiểm thử hậu điều kiện từ góc nhìn người dùng, quan sát rollback và quyền thao tác; kết quả demo chỉ chứng minh các nhánh được thử trong môi trường thử.

Bài tập

C8.1. Với `orchestrate_python.py bad fail`, dự đoán thứ tự các bước, status của helper ở từng bước, output cuối và nội dung `state.txt`. Sau `Popen` và `communicate`, chương trình lấy kết quả thực thi từ đâu? Vì sao nhánh `bad` vẫn báo thành công dù đã nhận kết quả lỗi, còn nhánh `checked` dừng?

C8.2. Với bản Go `checked slow`, hãy lần theo `READY`, `go`, lúc bắt đầu bộ đếm và kết quả `Wait`. Vì sao `TIMEOUT` khác exit status của helper, còn file vẫn chứa `partial` dù điều phối thoát 1? Nêu điều gì không thể kết luận về tiến trình cháu hoặc dịch vụ từ xa.

C8.3. Một tên môi trường do người dùng nhập là `prod; echo LEAK`. So sánh truyền nó như một phần tử argv với ghép vào chuỗi `sh -c`. Vì sao argv tách rời vẫn phải kiểm tra schema và quy tắc tùy chọn của utility? Nêu một nơi có thể làm lộ secret dù không dùng shell.

C8.4. Sau timeout, người vận hành định retry vô hạn bước ghi và xóa `partial` trước mỗi lần. Phân tích rủi ro nhân đôi, mất chứng cứ và chạy đồng thời; đề xuất điều kiện tối thiểu để thử lại, cùng phép kiểm chứng hậu điều kiện. Khi nào Bash vẫn là lựa chọn hợp lý cho một tác vụ tương tự?

Chương 9. Từ commit đến phục hồi một bản phát hành

Chương 6 cho ta một commit xác định mã nguồn; Chương 8 cho thấy chương trình điều phối có thể báo thành công dù hệ thống còn ở trạng thái từng phần. Phát hành một dịch vụ nối hai điều ấy bằng một chuỗi quyết định có chứng cứ: chọn commit, tạo artifact, kiểm thử chính artifact ấy, cho phép triển khai, chuyển lưu lượng, kiểm tra hành vi thật và chuẩn bị đường phục hồi. Mỗi mũi tên có thể thất bại độc lập. Một pipeline màu xanh không thay thế được câu hỏi người dùng đang nhận phiên bản nào và dữ liệu đã đổi ra sao.

9.1. Danh tính của bản được chạy

Commit định danh một trạng thái của repository, không định danh duy nhất binary hay image. Hai lần build từ cùng commit có thể khác byte vì dependency được giải lại, toolchain khác, timestamp, thứ tự đóng gói hoặc cờ build khác. Muốn giảm biến thiên phải khóa đầu vào, phiên bản compiler, base image, cấu hình build và môi trường; nếu đòi tái lập từng byte, phải đo bằng cách build độc lập rồi so digest, thay vì suy từ một file lock. Tính tái lập là thuộc tính của toàn quá trình và những đầu vào được kiểm soát, không phải lời hứa của tên phiên bản.

Artifact là tập byte đã được đóng gói để chạy. Digest tính từ byte cho phép kiểm tra ta đang dùng đúng nội dung đã kiểm thử; nhãn như `orders:1.8.4` chỉ là tên có thể bị trỏ lại nếu kho cho phép. Với image, descriptor chứa digest và kích thước để người nhận xác thực nội dung. Bản ghi provenance nối subject artifact với nguồn và cách build; nó giúp điều tra và kiểm tra chính sách, nhưng chỉ đáng tin trong phạm vi builder và quy trình tạo chứng cứ được tin cậy. Giữ riêng ba danh tính: commit nguồn, digest artifact và bản ghi triển khai ở môi trường. Khi quảng bá từ thử nghiệm sang production, dùng lại cùng digest; build lại từ cùng commit là một artifact mới, cần xét lại bằng chứng kiểm thử tương ứng.

9.2. Cổng kiểm thử và quyền phát hành

CI tích hợp thay đổi bằng build, kiểm tra và phản hồi sớm cho mỗi commit hoặc đề nghị hợp nhất. CD đưa một artifact đã qua cổng tới một hay nhiều môi trường; continuous delivery có thể dừng ở phê duyệt trước production, còn continuous deployment tự đi tiếp khi chính sách cho phép. Build tạo đầu ra; release là quyết định gắn đầu ra ấy với một phiên bản được phép dùng; deployment thay đổi cái đang chạy. Ba sự kiện có thể cách nhau nhiều ngày. Phân biệt chúng giúp biết chính xác cổng nào đã kiểm tra byte nào.

Các job là đồ thị phụ thuộc: kiểm tra tĩnh và unit test có thể chạy song song, nhưng kiểm thử tích hợp cần binary, database và cấu hình đã chuẩn bị; triển khai chỉ được dùng artifact mà các cổng đã kiểm tra. Unit test kiểm tra logic hẹp; integration test kiểm tra hợp đồng với thành phần thật hoặc mô phỏng có giới hạn; smoke test sau triển khai chỉ thử vài đường trọng yếu. Một test pass chứng minh những trường hợp đã chạy, không chứng minh không có lỗi trên dữ liệu production. Ghi input, phiên bản, kết quả và lý do bỏ qua cho từng cổng. Nếu thay artifact hoặc cấu hình có ảnh hưởng, không gắn nhãn “đã kiểm thử” từ lượt khác.

Môi trường triển khai có schema dữ liệu, cấu hình, quyền, secret, quota và lưu lượng riêng. Secret được cấp ở môi trường cần dùng và không nằm trong source, artifact hay log. Cổng phê duyệt production cần cho người duyệt thấy digest, khác biệt cấu hình, kết quả test, kế hoạch dừng và phương án phục hồi; nút bấm phê duyệt không tự kiểm tra những điều đó. Hai bản phát hành chạy đồng thời vào cùng môi trường có thể ghi đè nhau: dùng một khóa điều phối theo đích, xác nhận phiên bản hiện tại trước khi thay và định nghĩa rõ yêu cầu mới sẽ chờ hay hủy lượt cũ. Khóa pipeline không thay thế khóa ở hệ thống đích nếu còn công cụ triển khai khác.

9.3. Triển khai không đồng nghĩa sẵn sàng

Một Pod hoặc process đã chạy chỉ chứng minh scheduler và runtime khởi động được nó. Readiness phải phản ánh khả năng phục vụ đường yêu cầu cần thiết; một probe chỉ trả 200 từ bộ nhớ có thể xanh dù kết nối database hỏng. Trong Kubernetes, Pod chưa ready không nhận lưu lượng qua Service tương ứng, nhưng client đi đường khác hoặc yêu cầu đã bay tới vẫn cần xử lý riêng. Sau chuyển lưu lượng, kiểm tra từ phía client: phiên bản trả về, lỗi, latency và kết quả nghiệp vụ; gắn quan sát với digest hoặc release ID, không chỉ đếm process còn sống. Giới hạn thời gian quan sát và tiêu chí dừng phải được chọn trước khi thay đổi.

Rolling update thay dần instance cũ bằng mới, điều chỉnh số instance có thể thiếu và số được tăng thêm. Nó tiết kiệm tài nguyên hơn việc giữ hai cụm đầy đủ, nhưng cũ và mới cùng phục vụ trong một khoảng; schema, thông điệp và cache phải tương thích hai chiều trong thời gian đó. Blue-green chuẩn bị một tập mới rồi đổi tuyến lưu lượng; chuyển lại tuyến có thể nhanh nếu tập cũ còn khỏe, đổi lại phải trả thêm tài nguyên và giữ đồng bộ dữ liệu dùng chung. Canary gửi một phần lưu lượng tới bản mới, so cùng thời điểm với nhóm cũ rồi tăng dần. Nó giới hạn phạm vi ban đầu, nhưng mẫu nhỏ hoặc lưu lượng không đại diện có thể bỏ sót lỗi; sự kiện hiếm vẫn có thể gây hại ngay ở 1%. Chiến lược nào cũng cần xác định lưu lượng nền, phiên dài, queue, job nền và thao tác ghi không đi qua bộ cân bằng tải.

Rollback ứng dụng là đưa lưu lượng hoặc cấu hình chạy về artifact trước đó. Nó không tự đảo một migration database đã commit, hàng đợi đã phát, email đã gửi hay giá trị nghiệp vụ đã ghi sai. Một migration kiểu mở rộng, chẳng hạn thêm cột nullable trong khi phiên bản cũ vẫn đọc được bảng, thường cho đường quay về ứng dụng dễ hơn việc đổi tên hoặc xóa cột đang được bản cũ dùng. “Rollback database” là một hành động khác: lệnh `ROLLBACK` chỉ hủy thay đổi của giao dịch còn mở; sau `COMMIT` cần migration bù, khôi phục dữ liệu có kiểm soát hoặc một quy trình sửa riêng. Không chạy phép đảo cơ học trên dữ liệu sống khi chưa biết các ghi hợp lệ đã xen vào.

9.4. Sự cố xuyên suốt: bản chạy đã lùi, dữ liệu chưa lùi

Tình huống sau là dữ liệu giả định để suy luận, không phải log production hay benchmark. Dịch vụ `orders-api` đang chạy 1.8.3, build từ commit `c83`, artifact `A83` có digest `H83`. Lượt mới lấy commit `c84`, tạo `A84` với digest `H84` khác `H83`. Pipeline khóa dependency và toolchain, unit test pass; test tích hợp chỉ thử tạo đơn có tổng bằng 0 nên không bắt lỗi đổi đơn vị tiền. Artifact `A84` được đưa qua staging, nhưng production dùng dữ liệu và lưu lượng khác. Bản ghi release lưu `c84 → H84 → test-run T84 → production`, cùng người duyệt và thời điểm; dấu mũi tên chỉ quan hệ, không hàm ý các chuỗi ký hiệu này là digest SHA-256 thật.

Trước khi chuyển lưu lượng, migration thêm `receipt_token` nullable vào bảng `orders` rồi commit. 1.8.3 bỏ qua cột mới nên vẫn chạy với schema đã mở rộng. 1.8.4 có lỗi: đầu vào `amount_cents=1234` bị nhân thêm 100 trước khi ghi, thành `123400`. Bản canary nhận 5% lưu lượng. Trong một cửa sổ giả định, kiểm tra một đơn canary có giá trị ghi `123400` trong khi yêu cầu và sổ đối soát độc lập đều là `1234`; ba đơn khác cũng có chênh lệch. Readiness của Pod vẫn xanh vì endpoint probe không đọc đường ghi đơn. Không suy rằng mọi request trong 5% đều sai, cũng không suy tổng thiệt hại từ bốn mẫu; chúng đủ để dừng tăng canary và điều tra.

Trước hết đóng đường ghi của `H84` hoặc chuyển toàn bộ lưu lượng ghi về `H83`, tạm dừng lượt triển khai đang chạy và xác nhận không có worker/queue consumer `H84` tiếp tục ghi. Xác minh `H83` đáp ứng request mới với số tiền đúng và 1.8.3 tương thích schema có `receipt_token`. Giữ cột mới vì việc rút schema không cần cho phục hồi phục vụ; nó đã commit và có thể chứa dữ liệu hợp lệ. Sau đó khoanh tập bản ghi do `H84` tạo trong khoảng canary bằng release ID, thời gian, request ID và đối soát độc lập; sửa qua quy trình có phê duyệt, transaction và audit trail. Không chia mọi `amount_cents` cho 100: có thể có đơn hợp lệ do `H83` ghi xen vào. Hậu điều kiện gồm digest phục vụ hiện tại là `H83`, không còn đường ghi `H84`, đơn mới khớp nguồn đối soát và toàn bộ đơn bị ảnh hưởng đã được tìm và xử lý hoặc ghi rõ còn chờ. Một thông báo “rollback succeeded” chỉ chứng minh controller đã đổi cấu hình, không chứng minh bốn điều ấy.

Nếu migration đã xóa cột 1.8.3 cần, đưa `H83` trở lại sẽ không phục vụ được; phải khôi phục tính tương thích schema hoặc phát hành bản sửa tiến lên trước khi chuyển. Nếu yêu cầu từ `H84` đã gọi hệ thống thanh toán bên ngoài, việc đổi image không hủy giao dịch ở đó; cần truy trạng thái bằng khóa idempotency và quy trình bù theo hợp đồng của hệ thống đích. Vì vậy kế hoạch release phải nêu cả “quay lại code” lẫn “xử lý hậu quả đã commit”.

Bài tập

C9.1. Một pipeline build lại từ cùng commit khi đưa từ staging sang production, rồi chỉ so tên tag. Hãy chỉ ra mắt xích chứng cứ bị đứt và thiết kế bản ghi tối thiểu để chứng minh production chạy đúng artifact đã kiểm thử. Trường hợp nào hai lần build cùng commit có thể khác byte?

C9.2. Trong tình huống `orders-api`, unit test và readiness đều xanh. Vì sao hai tín hiệu ấy không bác bỏ lỗi `amount_cents`? Nêu phép kiểm tra hậu triển khai phát hiện sai lệch và giới hạn của mẫu canary nhỏ.

C9.3. So sánh rolling, blue-green và canary cho dịch vụ vừa nêu khi chỉ có đủ tài nguyên cho thêm một Pod. Chiến lược nào thực tế hơn, điều kiện tương thích dữ liệu nào phải giữ, và khi nào cần dừng dù tỷ lệ lỗi tổng thể chưa tăng rõ?

C9.4. Sau khi chuyển lưu lượng về `H83`, người trực định chạy `ROLLBACK` SQL và chia tất cả số tiền cho 100. Phân tích vì sao hai thao tác đó không chứng minh phục hồi. Đề xuất hậu điều kiện đo được cho phần ứng dụng, database và tác dụng phụ bên ngoài.

C9.5. Hai lượt release `R1` và `R2` cùng được phê duyệt vào production; `R2` hoàn tất trước nhưng `R1` ghi đè sau. Cổng phê duyệt thiếu điều kiện nào? Nêu cách điều phối theo môi trường và phép kiểm tra phiên bản ngay trước khi thay, đồng thời giải thích vì sao khóa ở CI chưa đủ nếu có người triển khai bằng công cụ khác.

Chương 10. Namespace, cgroup và tiến trình trong container

Chương 1 đặt tiến trình làm đơn vị chạy; Chương 2 đo tài nguyên mà các tiến trình tranh dùng. Container ghép hai cơ chế của nhân Linux quanh một nhóm tiến trình: namespace quyết định chúng nhìn thấy phần nào của hệ thống, còn cgroup tổ chức và kiểm soát mức dùng tài nguyên. Image ở Chương 9 là đầu vào bất biến để dựng môi trường chạy, không phải một tiến trình. Hiểu ba thực thể ấy giúp ta điều tra một dịch vụ chậm hoặc chết mà không gán mọi lỗi cho "container".

10.1. Một tiến trình có nhiều góc nhìn

Namespace gắn với tiến trình và bọc một loại tài nguyên bằng góc nhìn riêng. Trong PID namespace, cùng một tiến trình có thể là PID 1 khi nhìn từ bên trong và PID 3142 từ namespace tổ tiên; danh sách tiến trình nhìn thấy vì thế khác nhau. Trong network namespace, các interface, địa chỉ, route và socket thuộc một ngăn mạng riêng. `127.0.0.1` của tiến trình bên trong trỏ về loopback của ngăn ấy, không tự trỏ về host hay một container khác. Mount namespace thay bảng mount nhìn thấy; user namespace ánh xạ UID/GID; còn UTS và IPC tách các không gian tên tương ứng. Đây là phạm vi nhìn và định danh, không phải hạn mức CPU hoặc RAM.

Namespace mới phải được tạo hoặc tiến trình phải tham gia namespace đã có. `clone`/`clone3` có thể tạo tiến trình con ngay trong namespace mới theo cờ `CLONE_NEW*`; `unshare` tách một phần ngữ cảnh của tiến trình gọi. Trường hợp PID có điểm khác dễ bỏ sót: `unshare(CLONE_NEWPID)` chỉ đặt namespace PID cho những đứa con tạo sau đó, không chuyển PID của tiến trình gọi; đứa con đầu tiên thành PID 1. `setns` dùng file descriptor của namespace đã tồn tại để tham gia; với network namespace nó đổi góc nhìn mạng của tiến trình gọi, còn với PID namespace nó chỉ đổi nơi những đứa con sau này được tạo. `fork` thông thường kế thừa namespace, `execve` thay chương trình đang chạy mà không tự tạo namespace mới. Vì thế chuỗi tạo hoặc tham gia namespace, tạo con, rồi `execve` chương trình ứng dụng có thể dẫn tới PID 1 bên trong; chỉ gọi `execve` trên tiến trình hiện tại không tạo ra kết quả ấy.

Tạo hoặc tham gia PID/network namespace cần capability phù hợp, thường là `CAP_SYS_ADMIN` trong user namespace liên quan; cấu hình user namespace và chính sách host cũng ảnh hưởng việc cho phép. Lỗi `EPERM` là kết quả cần xử lý, không phải lý do tự nâng quyền trên máy đang phục vụ. Một phép quan sát chỉ đọc trên Linux là chạy `readlink /proc/$$/ns/pid` và `readlink /proc/$$/ns/net` trong shell, rồi chạy hai lệnh ấy trong một shell con `sh -c`; mặc định danh tính namespace của cha và con trùng nhau. Phép thử chỉ chứng minh lượt `fork`/`exec` ấy kế thừa góc nhìn, không chứng minh có thể tạo hay tham gia namespace, và `/proc` phải phản ánh đúng tiến trình đang xét.

Góc nhìn phải gắn với tiến trình đang xét. Từ host Linux có quyền phù hợp, `/proc/3142/status` có thể ghi `NSpid: 3142 1`, cho biết hai PID theo thứ tự các cấp namespace; `/proc/3142/ns/pid` và `/proc/3142/ns/net` cho biết danh tính namespace. Một số container có PID namespace riêng nhưng chia sẻ network namespace với host; cũng có thể chia sẻ PID namespace. Cấu hình runtime quyết định từng loại: loại namespace không khai báo có thể được kế

thừa. Không thể suy từ nhãn “container” rằng mọi namespace đều tách. PID 1 trong namespace còn là điểm cần xem khi xử lý tín hiệu và tiến trình con; một ứng dụng bỏ qua việc thu hồi con có thể tích lũy zombie dù host vẫn hoạt động bình thường.

Namespace không tạo tài nguyên từ hư không và không tự làm một sandbox hoàn chỉnh. Tiến trình vẫn dùng cùng nhân Linux với host; quyền, capability, seccomp, chính sách bảo mật nhân, mount và cấu hình runtime quyết định thêm bề mặt truy cập. Máy ảo thông thường chạy một nhân khách trên phần cứng ảo do hypervisor cung cấp. Ranh giới và chi phí vận hành khác nhau; trên máy để bàn, một hệ container Linux thậm chí có thể chạy bên trong máy ảo Linux. Vì vậy phải nói rõ đang xét ranh giới nào trước khi so sánh tính cô lập.

10.2. Cgroup v2 điều tiết tài nguyên

Cgroup v2 đặt tiến trình trong một cây phân cấp. Controller ở cha phân phối hoặc giới hạn tài nguyên cho các nhóm con; giới hạn ở nhóm con không vượt được ràng buộc của tổ tiên. Đây là cơ chế kiểm soát mức dùng, không thay đổi PID, route hay danh sách file mà tiến trình nhìn thấy. `cpu.weight` là trọng số cạnh tranh khi có nhóm cùng cần CPU, còn `cpu.max` là quota trên mỗi chu kỳ. Chẳng hạn `cpu.max` bằng `50000 100000` cho phép tối đa 50.000 microsecond CPU trong chu kỳ 100.000 microsecond: xấp xỉ nửa lõi trung bình khi luôn có việc, không hứa mỗi request sẽ hoàn tất trong 50 ms. Khi hết quota trước khi hết chu kỳ, tiến trình có thể phải chờ dù host còn CPU nhàn rỗi ngoài cgroup.

`cpu.stat` có `usage_usec`; khi CPU controller được bật, nó còn có `nr_periods`, `nr_throttled` và `throttled_usec`. Muốn quy chậm cho quota, đọc hai mẫu trên cùng cgroup và cùng khoảng thời gian, tính hiệu số, đồng thời so với thời điểm request chậm. Một mẫu sau sự cố không cho biết counter đã tăng khi nào. `memory.current` là mức dùng hiện tại, còn `memory.max` là giới hạn cứng. Khi không thể thu hồi đủ bộ nhớ dưới giới hạn, nhân có thể kích hoạt OOM trong cgroup. `memory.events.local` cho biết sự kiện tại cgroup ấy; `oom_kill` tăng nghĩa là đã có tiến trình thuộc nhóm bị OOM killer giết. `max` tăng chỉ nói đã chạm giới hạn, không đồng nghĩa đã có tiến trình chết. Bộ nhớ hiện tại có thể giảm mạnh sau khi nạn nhân bị giết, nên một ảnh chụp thấp không bác bỏ OOM vừa xảy ra. `memory.high` là ngưỡng thúc đẩy thu hồi và gây chậm; nó khác `memory.max` và không phải điểm kích hoạt OOM trực tiếp.

10.3. Image được thực thi thành container như thế nào

Image OCI gồm những thay đổi hệ thống tệp có thứ tự và cấu hình chạy như entrypoint, đối số, biến môi trường. Digest giúp nhận diện nội dung image đã lấy về, nhưng byte của image không tự tạo một namespace hay một cgroup. Runtime nhận bundle và cấu hình, chuẩn bị root filesystem, namespace, quyền và giới hạn, rồi khởi chạy chương trình. Container đang `running` là một tiến trình đã thực thi chương trình trong môi trường ấy; khi tiến trình chính dừng, runtime ghi trạng thái `stopped`. Image vẫn còn để dùng cho lượt chạy khác. Lớp ghi, volume và dữ liệu bên ngoài có vòng đời riêng, nên xóa hoặc chạy lại container không có nghĩa phục hồi dữ liệu như cũ.

Trạng thái `running` của runtime cũng không phải readiness của ứng dụng. Tiến trình có thể còn sống nhưng chưa bind socket, đang đợi DB, trả lỗi nghiệp vụ hoặc nghe sai địa chỉ. Việc publish port qua host hay qua bộ điều phối thêm một đường chuyển tiếp ngoài network namespace; thấy socket lắng nghe ở `127.0.0.1` bên trong chưa chứng minh client ngoài namespace đến được. Cần nối các phép quan sát từ tiến trình, route và socket tới phép thử request thực tế, như đã học ở Chương 4–5.

10.4. Một sự cố, bốn giả thuyết khác nhau

Tình huống giả định để tính toán, không phải output đo trên máy người học. Dịch vụ `orders` chạy trong một PID và network namespace riêng, dùng cgroup v2 `/service/orders`. Client nhận timeout xen kẽ; sau đó runtime báo một lần khởi động thất bại. Người vận hành có quyền đọc trạng thái từ host Linux. Trước hết họ gắn đúng PID host của lượt chạy và thời gian quan sát, tránh trộn counter của container cũ với lượt chạy mới. Với PID host 3142, `NSpid: 3142 1` và namespace `net:[4026533001]` khác host `net:[4026531992]`: đó là chứng cứ về góc nhìn PID và mạng khác nhau, chưa phải chứng cứ lỗi mạng. Từ góc nhìn network namespace của tiến trình, ứng dụng nghe `127.0.0.1:8080`; client ở namespace khác không thể dùng trực tiếp loopback ấy. Muốn chốt nhánh này phải kiểm tra đường publish/forward, địa chỉ bind, route và phép kết nối từ chính vị trí client. Nếu đường publish được cấu hình tới địa chỉ khác mà không có listener tương ứng, timeout có thể đến từ mạng; nếu TCP đã kết nối rồi nhận HTTP 500, lỗi nằm sau bước kết nối.

Trong một cửa sổ 10 giây của cùng lượt chạy, cấu hình là `cpu.max = 50000 100000`, `memory.max = 268435456` byte. Hai mẫu đọc từ cùng cgroup cho bảng sau; số là dữ liệu giả định, công thức hiệu số là phần cần kiểm chứng.

Counter	Đầu cửa sổ	Cuối cửa sổ
`cpu.stat usage_usec`	20.000.000	24.700.000
`cpu.stat nr_periods`	1.000	1.100
`cpu.stat nr_throttled`	40	120
`cpu.stat throttled_usec`	4.000.000	6.000.000
`memory.events.local max`	3	5
`memory.events.local oom`	0	1
`memory.events.local oom_kill`	0	1

Hiệu `usage_usec` là 4,7 triệu microsecond, tức trung bình 0,47 lõi trong 10 giây; quota cho tối đa xấp xỉ 0,5 lõi trung bình. Có 100 chu kỳ mới, 80 chu kỳ bị throttle, và `throttled_usec` tăng 2 triệu microsecond. Đây là bằng chứng quota đã tác động trong cửa sổ, phù hợp giả thuyết latency tăng; nó không cho biết request cụ thể nào chờ bao lâu, cũng không loại trừ I/O, khóa hoặc upstream chậm. Cần đặt timestamp request cạnh counter và, khi cần, đo thêm CPU pressure, hàng đợi và thời gian từng chặng. Không lấy tỷ số 80/100 làm “80% CPU đã dùng”: nó đếm chu kỳ có throttle, không đếm CPU time.

`memory.events.local max` tăng 2, `oom` tăng 1 và `oom_kill` tăng 1. Ít nhất một tiến trình trong cgroup này đã bị OOM killer giết trong cửa sổ; không thể từ counter ấy tự biết PID nào, request nào hoặc lỗi cấu hình nào làm bộ nhớ tăng. Phải đối chiếu thời điểm, danh tính tiến trình, log kernel/runtime và mức dùng trước khi bị giết. Nếu runtime báo exit 137, mã ấy riêng lẻ chỉ tương thích với việc tiến trình bị `SIGKILL` theo quy ước shell; OOM counter và log cùng thời điểm mới làm giả thuyết OOM mạnh hơn. Một lần khởi động sau đó thoát 2 với log `config missing`, trong khi `oom_kill` của cgroup mới không đổi: đó là bằng chứng của nhánh lỗi ứng dụng, không được gọi nhầm là OOM chỉ vì lượt trước từng OOM.

Để tái lập phép phân tích mà không gây OOM trên môi trường đang phục vụ, người học lấy hai bản chụp chỉ đọc của `cpu.max`, `cpu.stat`, `memory.max`, `memory.events.local` trên cùng đường cgroup và lưu thời điểm cùng PID/lượt chạy; rồi tính hiệu từng hàng như bảng. Trên Linux có quyền phù hợp, đọc `NSpid` và liên kết `/proc/<pid>/ns/{pid,net}`; quan sát route, listener từ đúng network namespace và thử kết nối từ vị trí client. Nếu đang ở cgroup namespace khác hoặc controller không được bật, đường dẫn/counter nhìn thấy có thể khác; ghi rõ ngữ cảnh hoặc dừng kết luận. Bảng này tái lập được phép tính, không tái lập một sự cố thật và không chứng minh cấu hình nào là an toàn cho production.

Bài tập

C10.1. Từ `NSpid: 3142 1`, namespace mạng khác host và listener `127.0.0.1:8080`, hãy nêu ba điều có thể kết luận và hai điều chưa thể kết luận về khả năng client kết nối. Phép quan sát nào phân biệt không có đường chuyển tiếp với ứng dụng đã nhận kết nối nhưng trả HTTP 500?

C10.2. Tính hiệu `usage_usec`, `nr_periods`, `nr_throttled`, `throttled_usec` trong bảng. Vì sao quota 0,5 lõi và 80 chu kỳ throttled ủng hộ nhưng chưa chứng minh CPU là nguyên nhân duy nhất của timeout?

C10.3. Phân biệt ý nghĩa `memory.events.local max`, `oom` và `oom_kill` tăng; đề xuất chứng cứ để gắn nạn nhân OOM với lượt chạy. Vì sao lần thoát 2 có log `config missing` không thể gộp với sự kiện OOM trước?

C10.4. Một image digest đã được kiểm thử nhưng runtime báo `running`, readiness vẫn đỏ. Giải thích vì sao cả digest lẫn trạng thái runtime không đủ chứng minh request được phục vụ. Nêu vai trò khác nhau của image, namespace và cgroup trong tình huống này.

Chương 11. Image, registry, Compose và vòng đời dữ liệu

Chương 10 cho thấy container đang chạy là một nhóm tiến trình với góc nhìn và giới hạn tài nguyên được cấu hình. Chương này đi ngược dòng tới thứ được phân phối để tạo tiến trình ấy, rồi theo dữ liệu đi qua nhiều lần chạy. Một câu hỏi vận hành thường bị trả lời sai là "đã đổi image, sao dữ liệu cũ vẫn ở đó?" Câu trả lời cần tách ba định danh: byte của image, lần chạy của container và nơi thực sự lưu dữ liệu.

11.1. Từ bản build tới registry

Builder đọc Dockerfile cùng build context, thực hiện các bước để tạo filesystem và cấu hình image. Theo mô hình OCI, manifest chỉ tới cấu hình và danh sách layer có thứ tự; descriptor có digest để người nhận kiểm tra đúng byte đã tham chiếu. Một layer ghi thay đổi so với lớp trước, không phải một máy ảo thu nhỏ. Lệnh `COPY` đưa dữ liệu của context vào kết quả build; file bị bỏ qua bởi `.dockerignore` không có trong context. Nếu đưa secret vào một bước build rồi xóa ở bước sau, không được mặc định rằng secret đã biến mất khỏi mọi layer hoặc cache. Cần cơ chế secret của builder và xác minh artifact cuối theo quy trình tương ứng.

Build cache tái dùng kết quả bước trước khi các đầu vào mà builder theo dõi được coi là không đổi. Khi một bước thay đổi, các bước phụ thuộc sau nó có thể phải chạy lại. Cache giúp tiết kiệm thời gian, nhưng cache hit không chứng minh chương trình đúng, dependency ngoài đã được cập nhật hay hai lượt build tạo byte giống nhau. Chương 9 đã yêu cầu ghi commit, toolchain, đầu vào và digest thực tế; ở đây cần thêm base image theo digest và các đầu vào build ngoài repository nếu muốn kiểm soát biến thiên. Build tạo image ở nơi build; push phân phối nội dung lên registry; pull lấy manifest và blob về nơi chạy. Ba hành động ấy khác nhau, và "đã pull" không có nghĩa container đang chạy đã được thay.

Registry lưu manifest và blob. Tag như `orders:2.1` là tên thuận tiện trỏ tới manifest và có thể được cập nhật; digest định danh nội dung manifest đã chọn. Khi registry trả về một index đa nền tảng, lựa chọn kiến trúc và manifest con còn phải được ghi nhận: cùng tag có thể dẫn tới nội dung khác trên hai nền tảng. Dùng digest của artifact đã kiểm thử để triển khai và xác nhận digest ở đích; vẫn cần đánh giá nguồn gốc, lỗ hổng, cấu hình và quyền vì đúng byte không đồng nghĩa hành vi an toàn. Một registry không sẵn sàng có thể làm lần pull mới thất bại dù container hiện hữu còn chạy trên node đã có image; trạng thái cache cục bộ và chính sách pull quyết định chi tiết.

11.2. Container là lần chạy; mount quyết định nơi ghi

Runtime tạo môi trường theo cấu hình, khởi chạy tiến trình và theo dõi trạng thái như `created`, `running`, `stopped`. Dừng tiến trình rồi khởi động lại cùng container có thể giữ writable layer của lần tạo ấy; xóa hoặc recreate container tạo lớp ghi khác. Thay image không tự sửa writable layer của một container đang tồn tại; phải xác định container nào thực sự đang phục vụ. Khi container bị xóa, dữ liệu chỉ nằm trong writable layer của nó không còn được giữ như một kho bền vững. Đặt database hoặc upload quan trọng ở đó là thiết kế dễ mất dữ liệu khi thay container.

Volume là vùng dữ liệu do Docker quản lý, được mount vào đường dẫn trong container. Named volume có định danh để một container khác có thể gắn lại; nó tồn tại độc lập với container cho tới khi bị xóa riêng. Bind mount thay bằng một đường dẫn cụ thể trên host của Docker daemon. Nó hữu ích cho mã nguồn hoặc cấu hình cần đọc từ host, nhưng bị ràng buộc vào đường dẫn, quyền và hệ điều hành của host ấy; khi dùng Docker Desktop, host của daemon có thể là Linux VM phía sau giao diện máy người dùng. Một bind mount có quyền ghi cho tiến trình trong container có thể sửa file host tương ứng, nên cấu hình chỉ cần đọc nên mount read-only. Cả hai loại mount che nội dung có sẵn tại điểm mount trong container; không nên thấy một thư mục "trống" rồi vội kết luận byte cũ đã bị xóa. Với empty named volume, Docker có thể sao chép nội dung sẵn trong image tại điểm mount vào volume theo mặc định; hành vi này khác việc chuyển dữ liệu từng được ghi ở writable layer của container cũ.

Đường dẫn ứng dụng không nói nơi lưu. `/var/lib/orders/orders.db` có thể thuộc writable layer, named volume, hoặc một thư mục host. Cần đọc cấu hình mount và danh tính container/volume rồi mới quyết định backup, migration hay xóa. Volume vẫn nằm trên một host hoặc backend cụ thể; "persist" không tự có nghĩa có bản sao, backup, giao dịch nhất quán hay khả năng chạy sang node khác. Database đang ghi cần phương pháp snapshot hoặc backup phù hợp với chính database, cùng phép restore thử. Xóa container, image và volume là ba hành động khác nhau; đặc biệt các tùy chọn xóa volume phải được xem như thao tác xóa dữ liệu.

11.3. Compose nối dịch vụ, mạng và nơi lưu

Compose file khai báo service, image hoặc cách build, port, network và volume cho một ứng dụng. `docker compose up` đối chiếu cấu hình để tạo hoặc thay container, mạng và volume cần thiết trên Docker Engine được chọn. Compose gán project để phân biệt tập tài nguyên; mặc định có mạng cho các service cùng project, chúng tìm nhau bằng tên service. Cổng host và cổng trong container là hai địa chỉ khác nhau: trong ví dụ dưới, client trên host dùng `127.0.0.1:18080`, còn tiến trình trong container phải nghe cổng 8080 trên địa chỉ có thể nhận kết nối từ mạng container. Đổi IP container sau recreate không nên buộc service khác sửa cấu hình; dùng tên service trong mạng chung. `localhost` trong service vẫn là chính network namespace của nó.

Đoạn Compose sau chỉ minh họa cú pháp. Image `orders:2.1` và thư mục `./config` là đầu vào giả định của dự án học, không có ứng dụng có thể chạy ngay trong repository này.

```
name: orders-case
services:
  orders:
    image: orders:2.1
    ports:
      - "127.0.0.1:18080:8080"
    volumes:
      - orders_data:/var/lib/orders
      - ./config:/etc/orders:ro
volumes:
  orders_data:
```

`orders_data` được mount tại đường ứng dụng ghi; `./config` là bind mount từ host, chỉ đọc trong container. `name: orders-case` cố định tên project trong ví dụ; chạy với project khác có thể tạo volume khác dù phần tên `orders_data` trong file giống nhau. Trước khi triển khai, phải xem cấu hình Compose đã được nội suy và kiểm tra image, mount, project, quyền, port cùng nơi daemon đang chạy. Đoạn này chưa có healthcheck; Compose thấy container chạy không chứng minh API hoặc database ready. `depends_on` mặc định giải quyết thứ tự khởi động, không tự kiểm tra ứng dụng đã sẵn sàng; điều kiện healthcheck có thể chờ một phép kiểm tra được khai báo, nhưng phép kiểm tra ấy vẫn chỉ chứng minh điều nó đo.

Compose hữu ích để mô tả và chạy ứng dụng trên một Docker Engine, kể cả Engine ở máy khác. File Compose tự nó không là bộ lập lịch nhiều node, không tự phân phối bản sao khi host hỏng và không tự nhân bản volume cục bộ. Nếu triển khai trên Swarm hoặc một hệ điều phối khác, phải đánh giá cơ chế scheduler, mạng nhiều host và storage của hệ đó thay vì suy từ cú pháp Compose. Chương 15–17 mới đi sâu vòng điều khiển và placement của Kubernetes.

11.4. Sự cố: cùng đường dẫn, ba nơi chứa dữ liệu

Tình huống sau là dữ liệu giả định để suy luận, không phải output Docker đã chạy. Ở lượt A, image `orders:2.0` không khai báo mount cho `/var/lib/orders`; ứng dụng ghi 41 bản ghi vào `orders.db` trong writable layer của container `orders-a`. Sau khi thay container bằng lượt B dùng image `orders:2.1`, đường dẫn ấy trống. Không có bằng chứng 41 bản ghi bị xóa khỏi một volume: chúng chưa từng ở volume; lớp ghi gắn với container A không phải dữ liệu được tự chuyển sang B. Nếu A còn tồn tại dưới dạng container đã dừng, có thể còn khả năng đọc lại lớp ghi; nếu A đã bị xóa, không được hứa khôi phục mà không có backup hoặc cơ chế lưu khác.

Ở lượt C, dự án dùng cấu hình Compose ở mục trước và ghi 41 bản ghi mới vào named volume `orders-case_orders_data`. Lượt D thay image `orders:2.1` bằng digest của bản `2.2`, recreate container nhưng vẫn gắn đúng volume ấy; ứng dụng nhìn thấy 41 bản ghi. Điều đó chưa chứng minh image `2.2` đã đọc dữ liệu đúng schema hoặc volume được backup. Nếu người vận hành đổi project name thành `orders-test`, Compose có thể gắn một volume mới `orders-test_orders_data`; đường dẫn trong container vẫn `/var/lib/orders` nhưng file cũ không xuất hiện. Volume của project trước có thể vẫn còn. Việc thêm bind mount `./empty-data:/var/lib/orders` vào cùng điểm mount cũng thay nơi nhìn thấy; không được đồng thời gắn hai nguồn vào cùng đích và đoán nguồn nào thắng. Chọn một cấu hình rõ ràng, kiểm tra `Mounts` thực tế và ngừng thay đổi trước khi điều tra.

Phép kiểm chứng tối thiểu bắt đầu bằng container ID, image digest và thời điểm recreate; tiếp đến là loại mount, `Source`, `Destination`, tên project và volume. So số bản ghi và checksum hoặc ID nghiệp vụ từ ứng dụng với bản backup độc lập, rồi thử đọc volume cũ bằng phương thức không ghi nếu cần. `docker compose config` giúp xem cấu hình sẽ áp dụng nhưng không chứng minh trạng thái runtime hoặc byte dữ liệu. Không chạy `down -v`, `volume prune` hay xóa thư mục host để "làm sạch" trước khi đã xác định chính xác nguồn dữ liệu. Trong môi trường thử riêng, có thể tái hiện ba lượt với dữ liệu vô hại và so trạng thái trước/sau; ở bản thảo này chưa có thực nghiệm

Docker nên không đưa output giả làm log thật.

Bài tập

C11.1. Một pipeline build và push `orders:2.2`; máy chủ vẫn chạy container tạo từ digest cũ. Phân biệt trạng thái registry, image cục bộ và tiến trình đang phục vụ. Cần đối chiếu những định danh nào để khẳng định bản mới đã nhận request?

C11.2. Giải thích vì sao 41 bản ghi ở lượt A không tự xuất hiện ở lượt B. Nếu container A chỉ dừng, câu trả lời khác gì trường hợp A đã bị xóa? Hành động nào phải tránh khi chưa có backup?

C11.3. Lượt D đổi image nhưng vẫn thấy 41 bản ghi; lượt kế tiếp đổi project name thì không thấy. Dùng volume identity và mount destination để giải thích hai hiện tượng. Vì sao một bind mount rỗng vào cùng đích không chứng minh volume cũ đã mất dữ liệu?

C11.4. Một Compose file có `depends_on` và hai service cùng mạng mặc định. Nêu điều nó đảm bảo, điều nó không đảm bảo về readiness, dữ liệu và phục hồi sau khi host hỏng. Đề xuất một phép kiểm chứng từ góc nhìn client và một phép kiểm chứng backup.

Lời giải

Chương 1. Từ chương trình đến tiến trình

C1.1. Artifact trên đĩa và lần thực thi đang chạy là hai thực thể khác nhau. Việc ghi đè một đường dẫn không chứng minh process image đã đổi. Cần kiểm tra PID và thời điểm khởi động, lệnh và artifact mà dịch vụ đang dùng, rồi so với định danh bản triển khai. Nếu ứng dụng có endpoint phiên bản, chỉ dùng nó khi biết endpoint phản ánh đúng build đang chạy. Khởi động lại phải theo quy trình triển khai, có kiểm tra sức khỏe và đường quay lui.

C1.2. Descriptor của tiến trình tham chiếu open file description đang mở, không tra cứu lại tên đường dẫn cho mỗi lần ghi. Đổi tên hoặc tạo tệp mới cùng tên không chuyển descriptor cũ. Cách xử lý là yêu cầu dịch vụ mở lại log bằng cơ chế được nó hỗ trợ, hoặc khởi động lại có kiểm soát sau khi bảo đảm log đã được thu thập và khả năng phục vụ không bị gián đoạn ngoài mức chấp nhận. Không xóa cưỡng bức dữ liệu trước khi xác định descriptor và nhu cầu lưu giữ.

C1.3. Sau `fork` thành công có hai tiến trình: cha và con. Nếu con gọi `execve` và lời gọi thất bại, chính nhánh con nhận giá trị lỗi và có thể báo lại; cha chỉ biết nếu có cơ chế thông báo hoặc chờ kết thúc. `execve` thay chương trình trong tiến trình gọi, nên không tăng số tiến trình. Trên đường thành công, lệnh sau `execve` trong chương trình cũ không chạy tiếp.

Chương 2. CPU, bộ nhớ và I/O: thời gian đi đâu?

C2.1. Theo mô hình cộng đã cho, độ trễ còn 20 + 60 = 80 ms. Trên hệ thống thật, thay đổi thời gian CPU có thể đổi lịch chạy, hành vi cache, mức cạnh tranh hoặc thời gian chờ của các yêu cầu khác; 80 ms chỉ là kết quả của giả định giữ mọi yếu tố kia cố định.

C2.2. Không gian địa chỉ ảo đo vùng địa chỉ mà tiến trình có thể dùng, không phải số trang vật lý đang cư trú. Cần phân biệt kích thước ánh xạ ảo và bộ nhớ cư trú; nếu phân tích áp lực chung, còn phải xét trang dùng chung, cache và cách cgroup tính bộ nhớ.

C2.3. Chưa đủ. Lời gọi ghi thành công không tự bảo đảm dữ liệu và metadata đã tới thiết bị bền vững. Khi tên tệp được tạo hoặc đổi, cần kiểm tra hợp đồng đồng bộ tệp và thư mục chứa mục tên đó, cùng hành vi hệ thống tệp thực tế. Không nên biến `fsync` trên tệp thành lời hứa tuyệt đối cho mọi bước của thao tác thay tên.

C2.4. `cpu.stat usage_usec` chia cho 60.000.000 µs cho 0,30 và 0,32 lõi trung bình; `nr_throttled` không tăng nên chưa có dấu vết quota CPU tại cgroup đã đo. `memory.current` cuối kỳ tăng 5 MiB, còn `memory.events high` và `oom` không tăng; chưa thấy chạm hai ngưỡng này, nhưng chưa loại trừ thu hồi ở máy hoặc cgroup tổ tiên. `io.pressure some total` chia cho 60.000.000 µs cho 2% và 30% thời gian có ít nhất một tác vụ nghẽn I/O. Cùng với p95 tăng, đó là lý do ưu tiên điều tra I/O. PSI không gắn thời gian chờ với từng yêu cầu hay thiết bị; bước tiếp theo là đo thời gian thao tác đọc theo yêu cầu, đối chiếu `io.stat` và tình trạng thiết bị trong cùng cửa sổ. Chưa thể quy toàn bộ độ trễ cho I/O.

Chương 3. Nhân Linux, hệ thống tệp, quyền và systemd

C3.1. Nhóm `atlas` có thể đọc tệp theo bit nhóm của `0640`, nhưng không có quyền tìm kiếm trên `/srv/atlas` khi thư mục là `0700 root:root`; phân giải đường dẫn có thể dừng ở thư mục đó với `EACCES`. Kiểm tra quyền từng thành phần và danh tính thực của dịch vụ; nếu hợp đồng cho phép, cấp quyền tìm kiếm trên thư mục cho đúng nhóm hoặc chuyển cấu hình sang cây thích hợp. Không đổi tệp thành `0666` hay dùng `chmod -R 777`: hai cách đó vừa không sửa quyền đi qua thư mục, vừa mở rộng truy cập.

C3.2. `Type=exec` xác nhận systemd đã thiết lập và thực thi được binary chính, theo hợp đồng của loại unit này. Nó không chứng minh ứng dụng đọc cấu hình thành công, mở cổng hay trả HTTP đúng. Xem nhật ký và trạng thái tiến trình chính, rồi gọi health endpoint hoặc phép thử chức năng từ đúng đường đi của người dùng. Một PID `active` không đủ thay cho phép thử khả năng phục vụ.

C3.3. Với binary thiếu, bước chuẩn bị/thực thi của systemd thất bại trước khi mã ứng dụng chạy. Với `openat` trả `EACCES`, binary đã chạy; nhân từ chối một thao tác tệp của tiến trình, ứng dụng phải ghi lỗi và chọn cách thoát. Nếu tiến trình thoát lỗi, `Restart=on-failure` có thể thử lại, rồi bị giới hạn tần suất khởi động. Tăng `RestartSec` chỉ đổi nhịp thử; đường dẫn hoặc quyền sai vẫn sai. Cần sửa đúng artifact, danh tính và quyền sau khi kiểm tra nguyên nhân.

Chương 4. Địa chỉ, định tuyến và TCP: một kết nối đi qua đâu?

C4.1. Client chọn gateway `10.20.1.1` làm next hop vì đích nằm ngoài prefix trực tiếp. Nó cần MAC của gateway trên liên kết cục bộ để đóng frame đầu tiên. MAC của server ở mạng xa không được ARP trực tiếp qua các router; mỗi chặng có frame và phép phân giải láng giềng riêng.

C4.2. Route `/24` cụ thể hơn route mặc định `/0`, nên được ưu tiên trong mô hình đã nêu. Trên Linux còn phải xét policy routing, bảng route, địa chỉ nguồn và namespace. `ip route get` trong đúng ngữ cảnh cho biết quyết định hiện tại, không bảo đảm gói thực sự qua được gateway.

C4.3. Kiểm tra route trả từ server tới địa chỉ nguồn mà server thực sự thấy, rồi kiểm tra ánh xạ NAT và lọc trên gateway hoặc các chặng về. Có thể đối chiếu capture hai phía của NAT theo thời gian và tuple trước/sau dịch. SYN-ACK đã rời server nhưng chưa tới client nghĩa là bắt tay TCP chưa hoàn tất ở client; HTTP chưa bắt đầu nên chưa thể quy nguyên nhân cho ứng dụng HTTP.

C4.4. Server trả tới `203.0.113.5:40001`, không gửi thẳng tới `10.20.1.10:51514`. Gateway cần ánh xạ còn hiệu lực và đường đưa gói về mạng trong để dịch đích trở lại. Ánh xạ NAT là trạng thái ở thiết bị dịch; `SYN-SENT` hay `ESTABLISHED` là trạng thái TCP của hai endpoint. Có mục NAT không chứng minh server đã chấp nhận kết nối.

C4.5. Giả thuyết mạnh nhất là server chỉ bind trên loopback, nên không có listener cho `198.51.100.20:443`. Vẫn kiểm tra quy tắc lọc có thể chủ động gửi RST. Sau khi sửa cấu hình bind trong môi trường được phép, xác nhận `ss -ltn` có đúng địa chỉ/cổng, thấy SYN, SYN-ACK, ACK và thử yêu cầu ứng dụng. `systemctl active` chỉ phản ánh trạng thái unit/process, không chứng minh socket nhận từ xa hay ứng dụng phục vụ đúng.

C4.6. SYN chiếm một vị trí nên sau `seq=1000`, byte hoặc số thứ tự tiếp theo của client là 1001; server báo `ack=1001`. Bốn byte ở 1001–1004 làm số tiếp theo là 1005, nên ACK cộng dồn là 1005. Nếu ACK mất rồi cùng đoạn dữ liệu được gửi lại với `seq=1001`, server vẫn báo `ack=1005` vì không xuất hiện byte mới liên tục; TCP nhận diện đoạn lặp và không giao bốn byte lần hai cho ứng dụng. Không thể suy từ một capture đơn lẻ rằng mất ACK là nguyên nhân duy nhất của retransmission.

Chương 5. Từ tên miền đến phản hồi: DNS, TLS, HTTP và reverse proxy

C5.1. Ghi tên và loại bản ghi mà ứng dụng hỏi, resolver nó dùng, thời điểm cùng TTL còn lại, rồi đối chiếu địa chỉ mà chính tiến trình đã kết nối. Có thể kiểm tra dữ liệu authoritative và cache của resolver để định vị chỗ sai. Một máy khác có cache, cấu hình resolver, đường đi hoặc thời điểm khác; kết quả của nó không đại diện cho client đang lỗi.

C5.2. Đối chiếu tên URL, SNI mà client gửi, chứng chỉ proxy chọn, tên trong chứng chỉ, chuỗi tin cậy, thời hạn và đồng hồ của client. `Host` là thông tin HTTP sau TLS; đổi nó không làm chứng chỉ đã chọn hợp lệ cho tên yêu cầu. Tắt xác minh biến lỗi rõ ràng thành một kết nối không được xác thực, không sửa binding hoặc chứng chỉ sai.

C5.3. Timeout trước TLS sau khi DNS có địa chỉ mới chỉ cho biết chưa có kênh TCP hoàn tất; xem SYN/SYN-ACK ở client và proxy. Certificate error chứng minh đã tới giai đoạn xác thực TLS, chưa có HTTP request hợp lệ; xem SNI và chứng chỉ. `502` chứng minh một HTTP response mang mã ấy đã tới client, cần log proxy xác định ai tạo và phản hồi upstream nào bị coi không hợp lệ. `504` cũng là HTTP response; đối chiếu thời gian proxy, upstream và nguồn tạo status. Không suy upstream chết từ `502`: ứng dụng có thể còn chạy nhưng trả header sai, đóng kết nối, hoặc chính upstream trả mã đó.

C5.4. Lần đầu có thể đã tạo đơn hàng dù proxy hết chờ, nên retry sang upstream khác có nguy cơ tạo bản ghi thứ hai. Chỉ retry tự động khi thao tác idempotent theo hợp đồng hoặc có khóa chống lặp và khả năng truy lại kết quả lần đầu; cần giới hạn số lần, thời gian và quan sát cả hai lần. Tăng timeout đổi ngưỡng proxy chờ, có thể giữ kết nối lâu hơn và che triệu chứng, nhưng không làm truy vấn dữ liệu hoặc phụ thuộc upstream nhanh hơn.

Chương 6. Git là đồ thị lịch sử, không phải thư mục sao lưu

C6.1. `HEAD` trỏ tới `B` với giá trị `1000`; index có snapshot `2000` tại lúc `git add`; working tree đã đổi thành `3000`. Commit ngay lấy index nên ghi `2000`, còn `3000` vẫn là thay đổi chưa stage. `git diff --cached` cho thấy `1000 → 2000`, `git diff` cho thấy `2000 → 3000`; stage lại rồi xem diff nếu định commit `3000`.

C6.2. `D` và `C` đều là con của `B`, không commit nào là tổ tiên của commit kia; dịch `main` thẳng tới `C` sẽ bỏ `D` khỏi lịch sử đi theo `main`. Merge sau giải conflict tạo `M` có cha thứ nhất `D`, cha thứ hai `C`; `main` trỏ tới `M`, feature vẫn trỏ tới `C`. Khi dừng vì conflict, `HEAD` còn ở `D`; index giữ stage 1 từ `B`, stage 2 từ `D` và stage 3 từ `C` cho đường dẫn xung đột, thay cho một bản đã giải ở stage thường.

C6.3. Rebase tạo `C'` có cha `D`, nên nhánh feature đi theo `A → B → D → C'`; `C'` có định danh mới vì parent của commit đã đổi. Nếu `C` đã được người khác dùng, dịch ref tới `C'` tạo lịch sử khác với lịch sử họ đang có và buộc phối hợp khi cập nhật nhánh chung. Merge giữ cả `C` lẫn `D` và ghi quyết định tích hợp ở `M`. Cả hai cách đều có thể cho snapshot sai; cần test timeout và retry trên kết quả cuối.

C6.4. `git revert -m 1 M` tạo một commit mới đảo tác động merge so với cha thứ nhất `D`; phải chọn mainline vì `M` có hai cha và "đảo merge" phụ thuộc cha nào được giữ làm nền. Xem diff vì nó không đơn giản đưa toàn kho về `B`, và merge lại nhánh cũ có thể không tự phục hồi thay đổi đã đảo. `git reset D` dịch ref `main` về `D`; nếu đã push `M`, lịch sử branch chia sẻ bị viết lại và bản sao của người khác có thể lệch. Cần review diff, xác nhận branch/ref và cha, chạy kiểm thử liên quan trên snapshot sau sửa, rồi kiểm tra trạng thái trước push.

Chương 7. Bash và ranh giới lỗi trong tự động hóa

C7.1. `tar -C $src` nhận hai word `/tmp/du` và `lieu` sau word splitting trong trường hợp giá trị đã cho; `tar -C "$src"` nhận một word `/tmp/du lieu`. Nếu biến chứa ký tự glob như `*`, lần mở rộng tên tệp có thể biến một word thành nhiều đường dẫn. `set -u` chỉ phát hiện một số lần dùng biến chưa đặt, không chặn tách word hay globbing. Lưu danh sách bằng mảng và truyền `"${files[@]}"` để bảo toàn ranh giới từng phần tử.

C7.2. Không có `pipefail`, status pipeline là 0 từ `tee`, nên `set -e` không thoát; `printf` in `BACKUP_OK` và script kết thúc 0. Có `pipefail`, pipeline trả 2 trong giả định này, `set -e` ở vị trí top-level khiến script thoát trước `printf`. Không nên chỉ dựa vào tương tác `set -e` vì nó có ngoại lệ ngữ cảnh; kiểm tra status tường minh rõ hơn. `tar` trả 0 chứng minh thao tác của `tar` đạt hợp đồng trả status của nó trong lần chạy ấy, không chứng minh restore hoàn chỉnh, nguồn nhất quán hoặc bản ghi đã bền vững qua mất điện.

C7.3. Hàm được gọi làm điều kiện `if`, nên `errexit` bị bỏ qua trong thân hàm ở ngữ cảnh này: `false` trả 1 nhưng `printf` vẫn chạy và trả 0. Hàm kết thúc 0, do đó in `done` rồi `ok`. Có thể viết `f() { if bước_đầu; then :; else return $?; fi; printf 'done\n'; }`, trong đó `bước_đầu` là lệnh thật cần kiểm tra. Cách khác là `bước_đầu || return $?` trong hàm, với chú ý lan truyền lỗi rõ ràng.

C7.4. Lần thứ hai vẫn tạo và kiểm tra file tạm; tới `ln`, tên `backup.tar` đã có nên GNU `ln` mặc định trả khác 0 và script thoát 5. `EXIT` trap xóa file tạm của lần ấy; hard link của bản đầu còn nguyên. `tar -tf` chỉ kiểm tra khả năng đọc cấu trúc archive; cần thử khôi phục và so dữ liệu, kiểm tra snapshot nhất quán và bền vững theo hệ thống tệp. `SIGKILL` không chạy trap, có thể để lại file tạm; nguồn bị sửa đồng thời có thể cho bản sao không nhất quán dù cleanup đúng. Việc phát hiện và xử lý hai rủi ro đó đòi hỏi cơ chế khác.

Chương 8. Python và Go: điều phối tác vụ, giữ đúng ranh giới lỗi

C8.1. `precheck` trả 0; `apply_fail` ghi `partial` rồi trả 7; `verify` đọc trạng thái ấy và trả 8. `Popen` khởi chạy helper, `communicate` chờ và thu dữ liệu; sau khi con kết thúc, `returncode` mang status cần xét. Nhánh `bad` ghi hai lỗi ra stderr nhưng bỏ qua status trong quyết định cuối, vẫn in `DEPLOY_OK` và thoát 0. Nhánh `checked` dùng status 7 để dừng với mã 1, nên không chạy `verify`; file vẫn chứa `partial`, vì dừng điều phối không đảo tác dụng phụ đã có.

C8.2. Helper phát `READY` sau khởi tạo và đợi `go`; cha chỉ bắt đầu bộ đếm 0,5 giây sau khi gửi `go`. `apply_slow` ghi `partial` trước khi ngủ hai giây; khi bộ đếm hết, cha hủy context, đợi `Wait` và trả nhãn `TIMEOUT`. Lỗi `Wait` sau khi kill không phải exit status 7 của nhánh `apply_fail`; `ctx.Err()` khi chủ động hủy là `context.Canceled`, nên không dùng nó một mình để suy ra nguyên nhân. Kill không hoàn nguyên file. `CommandContext` mặc định tác động tới tiến trình con trực tiếp; không thể kết luận tiến trình cháu hay yêu cầu từ xa đã dừng nếu không có cơ chế và quan sát riêng.

C8.3. Khi truyền `prod; echo LEAK` là một phần tử argv với API không gọi shell, dấu chấm phẩy là dữ liệu trong đối số; với `sh -c` và chuỗi ghép không được bảo vệ, nó có thể trở thành dấu tách lệnh. Dù dùng argv, utility vẫn có thể diễn giải giá trị bắt đầu bằng `-` như tùy chọn hoặc chấp nhận tên ngoài schema; cần xác thực và dùng ranh giới tùy chọn theo hợp đồng utility. Secret có thể lộ qua argv trong danh sách tiến trình, environment được kế thừa, hoặc log in nguyên command/exception.

C8.4. Timeout có thể đến sau khi hệ thống đích đã ghi nhưng trước phản hồi, nên retry vô hạn có thể tạo nhiều tác dụng, kéo dài quá thời gian cho phép và chồng lên lượt khác. Xóa `partial` mù quáng làm mất dấu vết và có thể xóa phần việc hợp lệ. Chỉ thử lại với điều kiện lỗi được phân loại là tạm thời, thao tác idempotent hoặc có khóa chống lặp, số lần/tổng thời gian hữu hạn và trạng thái hiện tại đã được đối chiếu; sau đó kiểm tra hậu điều kiện từ góc nhìn người dùng và log tương ứng. Bash vẫn hợp lý cho vài lệnh tuyến tính với ít trạng thái, đầu vào nhỏ, status được kiểm tra tường minh và môi trường chạy rõ ràng; không cần đổi ngôn ngữ chỉ vì script có một lệnh con.

Lời giải Chương 9

C9.1. Build lại tạo một tập byte mới; kết quả test staging chỉ gắn với digest cũ, còn tag có thể bị trỏ lại. Ghi commit, đầu vào và phiên bản build, digest artifact được test, test-run, approval, môi trường và digest thực sự được triển khai; promotion phải dùng lại digest đã test hoặc chạy lại cổng phù hợp cho artifact mới. Dependency không khóa, timestamp, toolchain hoặc cờ build khác có thể đổi byte dù commit giống nhau. Muốn kết luận tái lập từng byte phải so digest từ các lượt build độc lập có đầu vào kiểm soát.

C9.2. Unit test không phủ đơn giá trị 1234; readiness chỉ chứng minh điều kiện của probe, không kiểm tra phép ghi tiền. Gửi một đơn canary có đầu vào biết trước, đối chiếu bản ghi và sổ độc lập, gắn kết quả với `H84`; 123400 thay vì 1234 là bằng chứng lỗi cụ thể. Bốn mẫu cho phép dừng rollout nhưng không ước lượng chính xác tỷ lệ lỗi toàn hệ thống, nhất là khi luồng canary hoặc phân phối đơn hàng không đại diện.

C9.3. Blue-green đầy đủ cần giữ hai tập có năng lực phục vụ, nên thiếu tài nguyên nếu chỉ thêm được một Pod. Rolling có thể tăng một Pod rồi giảm dần bản cũ; canary có thể dùng chính Pod đó ở tỷ lệ lưu lượng nhỏ nếu bộ định tuyến hỗ trợ, sau đó mở rộng từng bước. Trong cả hai, 1.8.3 và 1.8.4 cùng truy cập schema nên việc thêm cột nullable phải giữ bản cũ đọc/ghi được; thông điệp, job và cache cũng cần xét. Sai một đơn tiền đã đối chiếu là lý do dừng ngay, dù tỷ lệ lỗi HTTP tổng thể vẫn đẹp.

C9.4. `ROLLBACK` SQL chỉ hủy giao dịch còn mở, trong khi migration và các đơn đã commit. Chia toàn bảng cho 100 phá các đơn đúng do `H83` ghi xen kẽ và không xử lý thanh toán bên ngoài. Hậu điều kiện ứng dụng: chỉ `H83` nhận ghi, xác nhận digest ở đường phục vụ và đơn mới đúng. Hậu điều kiện dữ liệu: tập đơn do `H84` ảnh hưởng được khoanh bằng chứng cứ, từng giá trị đối soát và sửa có audit; cột mới còn tương thích. Hậu điều kiện bên ngoài: mỗi thanh toán liên quan có trạng thái xác minh bằng ID, xử lý bù nếu hợp đồng cho phép, các trường hợp chưa biết được ghi riêng.

C9.5. Approval cho phép từng lượt nhưng không đảm bảo thứ tự và không xác nhận phiên bản đích tại lúc ghi. Cần serialize release theo môi trường và dùng kiểm tra điều kiện: chỉ thay nếu digest hiện tại đúng phiên bản mà lượt này dự kiến. Nếu `R2` đã thay trước, `R1` phải dừng hoặc được đánh giá lại, không ghi đè im lặng. Khóa CI chỉ quản lý các lượt đi qua CI; công cụ hoặc người triển khai trực tiếp vẫn có thể thay đích, nên hệ thống đích cũng phải kiểm tra danh tính bản hiện tại và quyền thao tác.

Lời giải Chương 10

C10.1. `NSpid` cho thấy cùng tiến trình có PID 3142 ở góc nhìn tổ tiên và PID 1 ở góc nhìn bên trong; danh tính network namespace khác host; listener chỉ bind loopback của ngăn mạng đang quan sát. Chưa thể kết luận client có hay không có đường publish/forward, cũng chưa biết request đã tới ứng dụng. Kiểm tra cấu hình chuyển tiếp và socket/route ở đúng hai đầu, rồi thử từ vị trí client: nếu TCP không thiết lập, điều tra đường mạng và listener; nếu có HTTP 500, kết nối TCP và tầng HTTP đã hoạt động đủ để có phản hồi, tiếp tục xem proxy/upstream và log ứng dụng. HTTP 500 riêng nó chưa chỉ rõ tiến trình nào sinh lỗi.

C10.2. Hiệu lần lượt là 4.700.000 microsecond CPU, 100 chu kỳ, 80 chu kỳ throttled và 2.000.000 microsecond throttling; mức dùng trung bình là 4,7/10 = 0,47 lõi. `50000/100000 = 0,5` lõi là trần trung bình theo chu kỳ. Throttling trong 80 chu kỳ cho thấy quota thực sự chặn công việc ở cửa sổ đo, nhưng không đo thời gian chờ của từng request và không loại trừ I/O, khóa hay upstream. Cần đối chiếu cửa sổ metric với trace/latency từng chặng trước khi quy nguyên nhân.

C10.3. `max` tăng 2 nghĩa là hai lần mức dùng chạm giới hạn; `oom` tăng 1 ghi nhận tình huống cấp phát sắp thất bại có xét OOM; `oom_kill` tăng 1 xác nhận một tiến trình trong cgroup bị OOM killer giết. Muốn xác định nạn nhân cần ghép timestamp, PID/lượt chạy, log kernel và runtime; một số đo `memory.current` sau sự kiện có thể đã giảm. Lần khởi động sau thoát 2 với log `config missing` và counter OOM của cgroup mới không đổi thuộc nhánh lỗi cấu hình ứng dụng; sự kiện OOM cũ không tự giải thích lần thoát này.

C10.4. Digest nhận diện byte image đã kiểm thử, không chứng minh cấu hình runtime, dependency ngoài, mạng hay dữ liệu hiện thời đúng. `running` chỉ xác nhận chương trình đã được thực thi và chưa dừng; readiness đỏ cần phép thử ứng dụng cụ thể, ví dụ bind, phụ thuộc và request đại diện. Image cung cấp root filesystem và tham số mặc định; namespace đặt góc nhìn tài nguyên của tiến trình; cgroup giới hạn/phân phối tài nguyên. Bất kỳ tầng nào cũng có thể góp phần làm request thất bại, nên cần quan sát đúng tầng rồi mới kết luận.

Lời giải Chương 11

C11.1. Build tạo artifact, push đưa manifest/blob lên registry, pull đưa nội dung về máy chạy; các bước ấy không tự thay tiến trình. Ghi commit nguồn, digest manifest đã kiểm thử, digest image được dùng tạo container mới, container ID, thời điểm chuyển lưu lượng và kết quả request từ client. Tag `orders:2.2` có thể trỏ lại nên tên tag riêng lẻ không đủ. Nếu máy chủ vẫn phục vụ container từ digest cũ, bản mới chưa được chứng minh là đang nhận request dù registry đã có nó.

C11.2. Lượt A ghi trong writable layer riêng của container A. B được tạo mới từ image nên có lớp ghi mới, không tự nhận dữ liệu từ A. Nếu A chỉ dừng và còn container, lớp ghi có thể còn đọc được để điều tra; nếu A bị xóa, không được hứa khôi phục từ lớp ghi ấy. Tránh xóa container cũ, chạy `down -v`, prune hoặc ghi đè đường dẫn nguồn trước khi xác định dữ liệu và có bản sao đã thử restore.

C11.3. D vẫn gắn `orders-case_orders_data` tại `/var/lib/orders`, nên đổi byte image không thay byte của named volume. Project `orders-test` tạo tài nguyên tên khác theo project; ứng dụng nhìn cùng đường dẫn nhưng có thể đọc volume mới. Bind mount rỗng đặt tại `/var/lib/orders` che nội dung ở điểm mount; volume cũ có thể vẫn tồn tại, chỉ không còn là nguồn dữ liệu hiện được gắn. Xác minh loại mount, `Source`, `Destination`, tên volume và đọc dữ liệu cũ theo cách không ghi trước khi sửa cấu hình.

C11.4. Mạng mặc định cho các service cùng project đường kết nối và tên để tìm nhau; `depends_on` mặc định sắp thứ tự khởi động, không đảm bảo API hay database đã ready. Ngay cả healthcheck chỉ bao phủ phép đo được khai báo. Compose file cũng không tự backup volume hoặc tự lập lịch bản sao sang host khác khi host chết. Gửi request đại diện từ đúng vị trí client và kiểm tra dữ liệu trả về; tạo backup nhất quán của dữ liệu thật rồi thử restore trong môi trường riêng, so ID và checksum hoặc hậu điều kiện nghiệp vụ.